

Apna Budget

Project Team

Ahmad Hassan	21I-0403
Muhammad Faraz Rashid	21I-0659
Muhammad Affan	21I-0474

Session 2021-2024

Supervised by

Ms. Marium Hida



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1: SMS and Notification Reading For Automated Transactions	2
1.3.2	Module 2: Uploading Bills and Receipts via Images	2
1.3.3	Module 3: Planning and Forecasting	3
1.3.4	Module 4: Final Tweaks And Additional Features	3
1.4	User Classes and Characteristics	3
2	Project Requirements	5
2.1	Use-case	5
2.2	System Sequence Diagrams	18
2.2.1	SSD 1: User Registration	18
2.2.2	SSD 2: Login with Email/Password	19
2.2.3	SSD 3: Login with Google	20
2.2.4	SSD 4: Password Reset	21
2.2.5	SSD 5: Automatic SMS and Notification Parsing for Transactions	22
2.2.6	SSD 6: Manual Transaction Entry	25
2.2.7	SSD 7: View Transactions	26
2.2.8	SSD 8: View Budgets	27
2.2.9	SSD 9: Create Budgets for Different Transaction Categories	28
2.2.10	SSD 10: Create New Categories	29
2.3	System Sequence Diagrams	30
2.3.1	SD 1: User Registration	30
2.3.2	SD 2: Login with Email/Password	32
2.3.3	SD 3: Login with Google	34
2.3.4	SD 4: Password Reset	35
2.3.5	SD 5: Automatic SMS and Notification Parsing for Transactions	36
2.3.6	SD 6: Manual Transaction Entry	38
2.3.7	SD 7: View Transactions	40

2.3.8	SD 8: View Budgets	41
2.3.9	SD 9: Create Budgets for Different Transaction Categories	43
2.3.10	SD 10: Create New Categories	44
2.4	Functional Requirements	45
2.4.1	Module 1: SMS and Notification Reading For Automated Trans- actions	45
2.4.2	Module 2: Uploading Bills and Receipts via Images	47
2.4.3	Module 3: Planning and Forecasting	48
2.4.4	Module 4: Final Tweaks And Additional Features	49
2.5	Non-Functional Requirements	50
2.5.1	Reliability	54
2.5.2	Usability	55
2.5.3	Performance	55
2.5.4	Security	55
2.6	Domain Model	55
3	System Overview	57
3.1	Architectural Design	57
3.2	Design Models	59
3.3	Data Design	63

List of Figures

2.1	Use Case Diagram of Apna Budget	6
2.2	SSD for User Registration	19
2.3	SSD for Login with Email/Password	20
2.4	SSD for Login with Google	21
2.5	SSD for Password Reset	23
2.6	SSD for Automatic SMS and Notification Parsing for Transactions	24
2.7	SSD for Manual Transaction Entry	26
2.8	SSD for View Transactions	27
2.9	SSD for View Budgets	28
2.10	SSD for Create Budgets for Different Transaction Categories	29
2.11	SSD for Create New Categories	30
2.12	SD for User Registration	31
2.13	SD for Login	33
2.14	SD for Login with Google	35
2.15	SD for Password Reset	36
2.16	SD for Automatic SMS and Notification Parsing for Transactions	37
2.17	SD for Manual Transaction Entry	39
2.18	SD for View Transactions	41
2.19	SD for View Budgets	42
2.20	SD for Create Budgets for Different Transaction Categories	44
2.21	SD for Create New Categories	45
2.22	Domain Model of Apna Budget	56
3.1	Architecture Diagram of Apna Budget	58
3.2	Component Diagram of Apna Budget	59
3.3	Activity Diagram of Apna Budget	60
3.4	Package Diagram of Apna Budget - Frontend	61
3.5	Package Diagram of Apna Budget - Backend	62
3.6	Class Diagram of Apna Budget - Frontend	63
3.7	Class Diagram of Apna Budget - Backend	63
3.8	Data Model Diagram of Apna Budget	64

List of Tables

Chapter 1

Introduction

This is a report regarding our FYP scope document. This is a development based project. Our goal is to produce a cross-platform budgeting mobile application targeting the general population of Pakistan. With a heavy economic decline we are producing an application to provide better financial awareness by helping people budget, plan, visualize, and forecast thus making them more aware of their financial situation and empowering them. The unique selling point of our application is: the automated SMS and Notification reading for detecting transactions and the integration of AI into our application. We are not dealing with any sort of transactions or digital wallets. We are just providing an all in one single application where user can see how and where his/her money flows. ?.

1.1 Problem Statement

Pakistan is currently facing one of the worst economic crisis in it's history, which is causing many people to struggle with managing their finances. Many people lack financial knowledge. This is leads to people making bad financial decisions, and falling in debt, and having fewer savings/investments. Popular applications in Pakistan like JazzCash and EasyPaisa, which cover 56on transactions and offer quite limited to zero features for managing money. For instance, JazzCash only shows the last 20 transactions from the past year. Other budgeting apps like Mint and You Need A Budget (YNAB) require users to manually enter every transaction, which is quite tedious and somewhat inconsistent as well. People are forced to switch between different apps to track their spending and income, which makes financial management harder and worse user experience. There is a huge gap and thus we are fulfilling these requirements and providing such features all in one place.

1.2 Scope

Our application is focused on the Pakistani market. Our application will not be focused on transactions. We will not be dealing with any money or wallets. Furthermore, our application will be focused on basic predictions

We will focus on income and spending graphs and trends of user activities. From that we'll generate predictions and suggest appropriate actions. Our application will not be focused on investments.

1.3 Modules

We are building a client mobile application that will help users in Pakistan manage their finances effectively. Our four main modules are:

1.3.1 Module 1: SMS and Notification Reading For Automated Transactions

This module focuses on user authentication as well as the automatic sms and notification parsing to enter transactions. It includes secure login ability, manual transaction entry, and the ability to categorize these transactions

1. User Authentication
2. SMS and Notification Reading for Automatic Transaction Retrieval
3. Ability to Manually Add Transactions
4. Ability to Categorize Transactions
5. Creating Budgets for Different Transaction Categories
6. Data Encryption and Masking for Sensitive Information

1.3.2 Module 2: Uploading Bills and Receipts via Images

In this module users will be able to upload images and screenshots of receipts and bills, which will also be automatically processed and categorized just like with the SMS and notifications feature. It also includes sorting and filtering, along with notifications related to categorization, budgets, and transactions.

1. Uploading Images and Screenshots for Automatic Transaction Processing
2. Visualization of Categories and Spending
3. Sorting and Filtering Categories
4. Notifications for Categorization, Budgets, and Transactions

1.3.3 Module 3: Planning and Forecasting

This module is for financial planning and forecasting. Users will be able to create budgets, visualize spending, set financial goals, and receive reminders for important financial tasks. It also provides scenario planning and forecasting tools to help with future planning

1. Ability to Set Plans and Goals
2. Planning Scenarios Generation
3. Visualization of Plans
4. Visualization of Goals

1.3.4 Module 4: Final Tweaks And Additional Features

This module includes additional features that will improve the overall functionality of the application, such as:

1. Stock Visualization
2. Comparison of Different Insurance, Savings, and Retirement Plans
3. Tax Template
4. Reminders for Bill Payments, Partial Payments, Cash Withdrawals, and Other Manual Reminders

1.4 User Classes and Characteristics

1. Introduction

User class	Description
Consumer	Our main user is the consumer of this application will be the the general population of Pakistan. They will be able to create an account, and go through their transactions, and track their expenses and budget

Example: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case

For the the project apna budget, we have created the following use case diagram. We mainly have one user, the consumer user which is able to perform his/her actions. An admin professional is only able to access server logs, and modify data in the database directly.

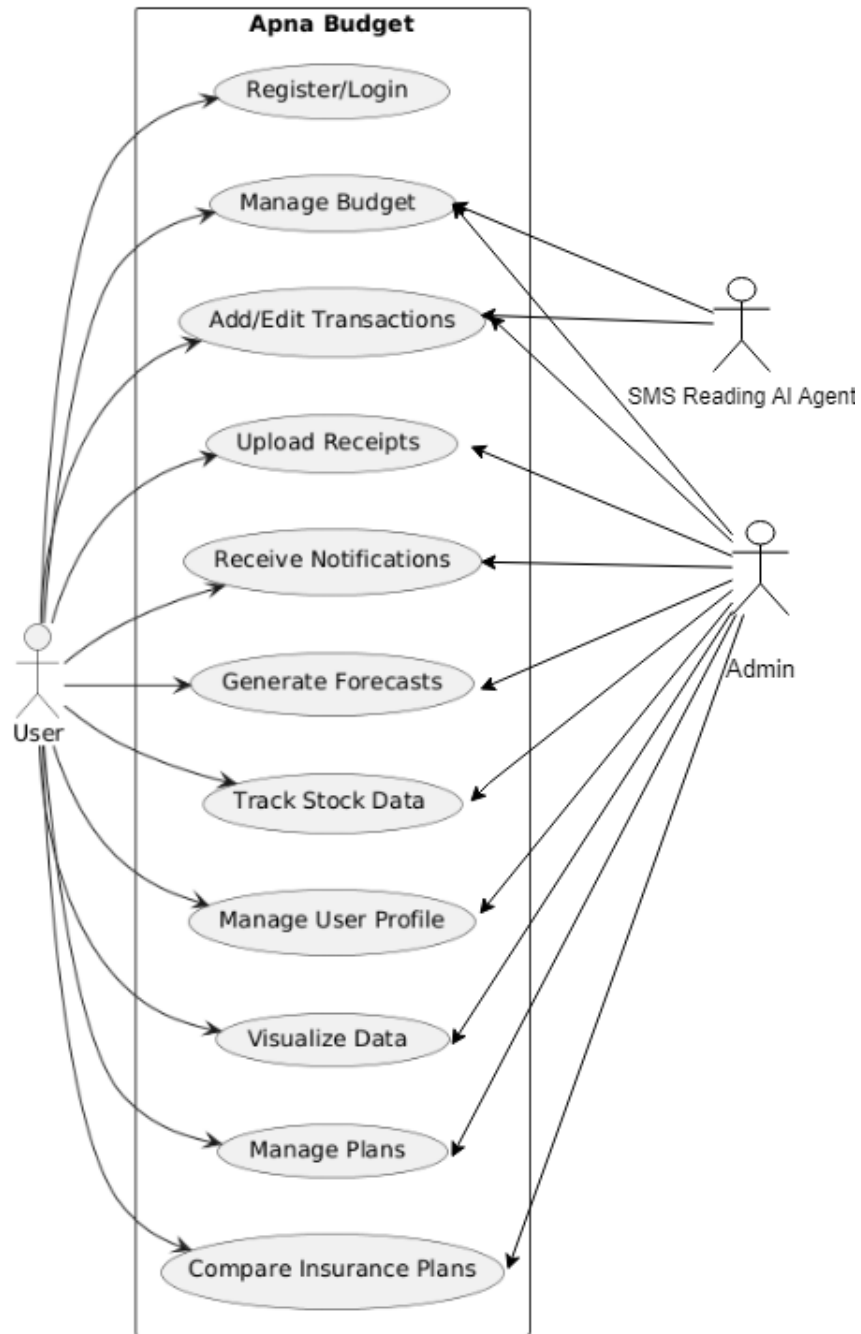


Figure 2.1: Use Case Diagram of Apna Budget

Use Case 01: User Registration

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user has installed the application.
- The user has a valid email address and/or mobile number.

Postconditions:

- Success: The user successfully created an account and is logged in to the app.
- Failure: The user fails to create an account due to incorrect input or system errors.

Trigger: The user selects the "Create Account" button on the initial screen.

Main Success Scenario:

Actor Action	System Response
The user selects "Create Account."	The system prompts the user to enter a valid email address and password.
The user inputs the required details.	The system sends an OTP to the user's email or mobile number for verification.
The user enters the OTP to verify their account.	The system confirms the OTP and creates the account.
	The user is logged in and taken to the main dashboard.

Extensions:

E1: Invalid Email Format:

If the user enters an invalid email, the system displays an error message.

E2: OTP Expired:

If the user delays entering the OTP, the system will offer to resend the OTP.

E3: Registration Fails:

If the system fails to create the account (e.g., network issues), it will inform the user and ask them to retry.

Use Case 02: Login with Email/Password

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user has a user account in the system.
- The user has a proper internet connection.

Postconditions:

- Success: The User is successfully logged in the system.
- Failure: Error message is displayed to the user.

Trigger: The user clicks on Login to get to the Login page.

Main Success Scenario:

Actor Action	System Response
The user selects "Log In."	The system prompts the user to enter their email and password.
The user inputs their credentials.	The system validates the credentials.
	The system sends an OTP to the user's email or mobile number for verification.

The user enters the OTP to verify their account.	The system confirms the OTP.
	If valid, the user is logged in and taken to the main dashboard.

Extensions:

E1: Invalid Email Format:

If the user enters an invalid email, the system displays an error message.

E2: Login Fails:

If the system fails to log in the user (e.g., network issues), it will inform the user and ask them to retry.

E3: Invalid Email or Password:

If the system fails to login due to wrong credentials, it will inform the user and ask them to retry.

Use Case 03: Login with Google Account

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user has a user account in the system.
- The user has a proper internet connection.

Postconditions:

- Success: The User is successfully logged in the system.
- Failure: Error message is displayed to the user.

Trigger: The user clicks on Login with Google to get to the Login page.

Main Success Scenario:

Actor Action	System Response

The user selects "Log In with Google"	The system connects to Google and gets a token.
	The system shows the user the result of the Login response.
	The System takes the user to the main dashboard.

Extensions:

E1: Login Fails:

If the system fails to connect (e.g., network issues or it can't validate token), it will inform the user and ask them to retry.

Use Case 04: Password Reset

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user has a user account in the system.
- The user has a proper internet connection.

Postconditions:

- Success: The User has successfully changed the password.
- Failure: Error message is displayed to the user.

Trigger: The user clicks on Reset Password.

Main Success Scenario:

Actor Action	System Response
	The system prompts the user to enter email for account verification. The user requests to “Reset Password.”
The user enters the email and clicks the “Send Email” button. The system checks the email in the database and sends a verification email.	
The user checks their email and clicks: “Verify.” The system takes the user to the Forget Password page.	
The user enters new Password and confirms Password entries.	The user clicks on submit.
	The system shows the Password changed message.

Extensions:

E1: New Password and Confirm Password mismatch:

If the user enters mismatched passwords, the system displays an error message.

E2: Reset Fails:

If the system fails to reset the password (e.g., network issues), it will inform the user and ask them to retry.

Use Case 05: Automatic SMS and Notification Parsing for Transactions

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user has granted SMS and notification access permissions to the app.
- The user has received an SMS or notification regarding a financial transaction.

Postconditions:

- Success: The system successfully detects and logs the transaction details from the SMS or notification.
- Failure: The system fails to detect SMS or finds it ambiguous based on category.

Trigger: The user receives a transaction notification (e.g., from a bank or financial app).

Main Success Scenario:

Actor Action	System Response
The user receives an SMS or notification for a transaction. The system detects the SMS or notification.	
	The system parses the message and extracts relevant transaction details (e.g., amount, merchant, date).
	The system categorizes the transaction (e.g., groceries, bills) and adds it to the user's transaction history.

	The system updates the user's budget and displays the updated balance.
--	--

Extensions:

E1: Permission Denied:

If the user has not granted SMS or notification permissions, the system prompts the user to enable these permissions.

E2: Ambiguous or Unrecognized Message:

If the system cannot recognize the message format or category, it prompts the user to manually categorize the transaction.

Use Case 06: Manual Transaction Entry

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user is logged in.
- The user wants to add a transaction that wasn't automatically detected.
- The user wants to add new transactions on their own.

Postconditions:

- Success: The user successfully adds a new transaction, and it is reflected in their budget.
- Failure: The transaction is not added due to errors or incomplete details.

Trigger: The user selects the option to manually add a transaction.

Main Success Scenario:

Actor Action	System Response

The user selects the "Add Transaction" button. The system prompts the user to enter details (e.g., amount, date, category).	
The user fills in the required fields and submits the transaction.	
	The system saves the transaction and updates the user's budget.

Extensions:

E1: Incomplete Information:

If the user leaves required fields blank, the system prompts the user to complete the missing information.

Use Case 07: View Transactions

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user is logged in.

Postconditions:

- Success: The user successfully sees the recent transactions data.
- Failure: The user is not able to see the transactions due to app crash.

Trigger: The user gets the notifications of the transaction.

Main Success Scenario:

Actor Action	System Response

The user opens up the app. The system displays the homepage which shows recent transactions.	
The user clicks “See all”.	
	The system shows all transactions related to different categories in detail.

Extensions:

E1: Application Failure:

If the system fails to Load due to some bug, it will inform the user.

Use Case 08: View Budgets

Scope: Apna Budget**Primary Actor:** User**Supporting Actor:** System**Preconditions:**

- The user is logged in.

Postconditions:

- Success: The user successfully views the Budgets in different categories.
- Failure: The user is not able to view budgets due to app crashes.

Trigger: The user gets the notification to see their budgets.**Main Success Scenario:**

Actor Action	System Response
The user opens up the app.	

The system displays the homepage which shows budgets.	
The user clicks “See all”.	
	The system shows all budgets related to different categories in detail.

Extensions:

E1: Application Failure:

If the system fails to Load due to some bug, it will inform the user.

Use Case 09: Creating Budgets for Different Transaction Categories

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user is logged in.
- The user has transaction categories to create budgets for.

Postconditions:

- Success: The system creates budgets for specific transaction categories and tracks expenses against them.
- Failure: Budget creation fails due to missing or incorrect information.

Trigger: The user selects "Create Budget" from the budget settings menu.

Main Success Scenario:

Actor Action	System Response

The user selects the "Create Budget" button. The system prompts the user to select a transaction category and enter a budget amount.	
The user provides the necessary information. The system creates the budget and tracks future transactions in that category.	
	The system displays visual indicators (e.g., charts, graphs) for progress against the budget.

Extensions:

E1: Category Not Selected:

If the user does not select a category, the system prompts them to choose one before creating the budget.

Use Case 10: Create New Categories

Scope: Apna Budget

Primary Actor: User

Supporting Actor: System

Preconditions:

- The user is logged in.

Postconditions:

- Success: The system creates the categories for the user.
- Failure: The system gives the errors or app crashes.

Trigger: The user selects "Create New Category" on the home screen.

Main Success Scenario:

Actor Action	System Response
The user selects the "Create Category" button. The system prompts the user to name a transaction category.	
The user writes the name of the category.	
	The system prompts the user to name a transaction category.

Extensions:

E1: Category Creation Failure:

If the system fails to save a new category into db due to some bug, it will inform the user.

2.2 System Sequence Diagrams

2.2.1 SSD 1: User Registration

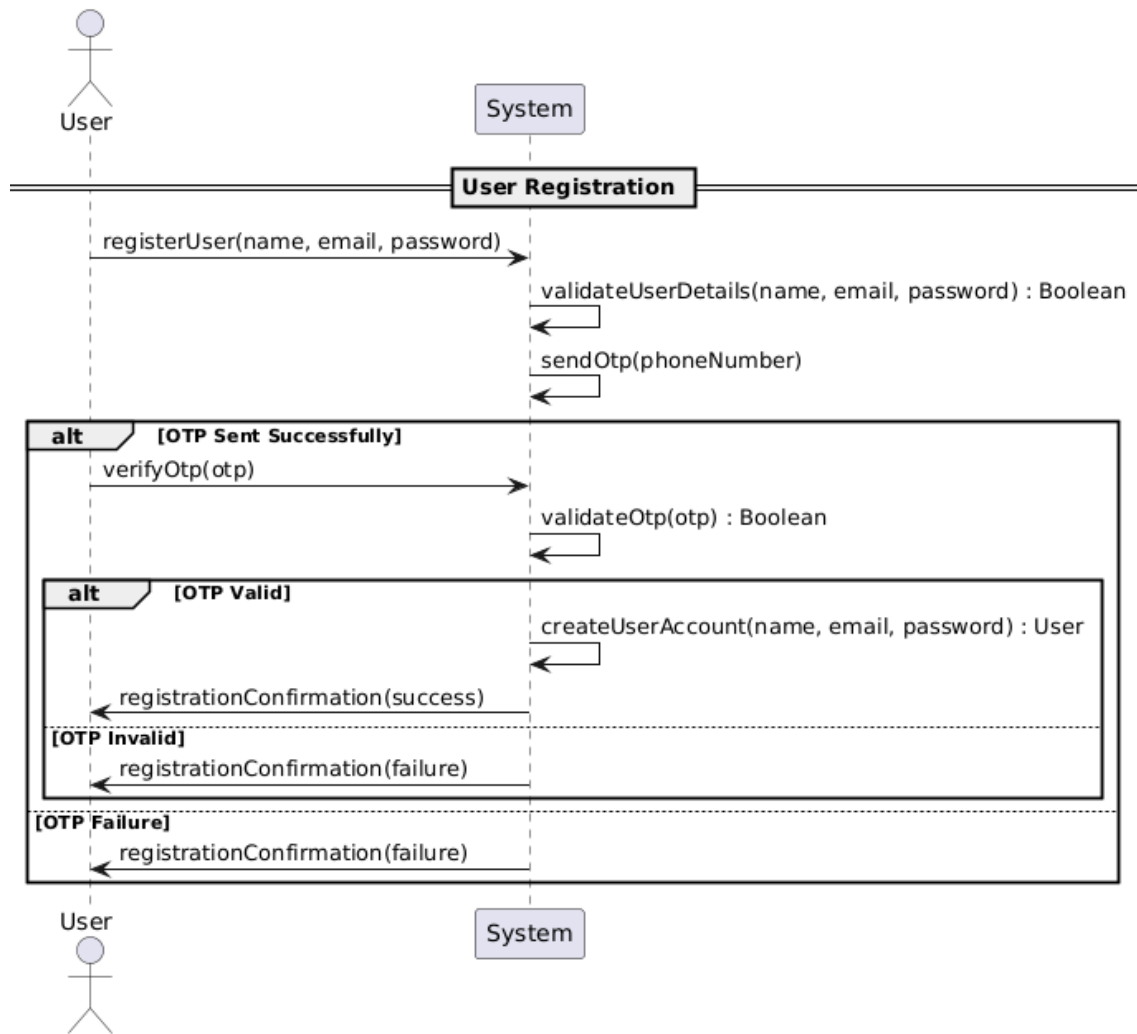


Figure 2.2: SSD for User Registration

2.2.2 SSD 2: Login with Email/Password

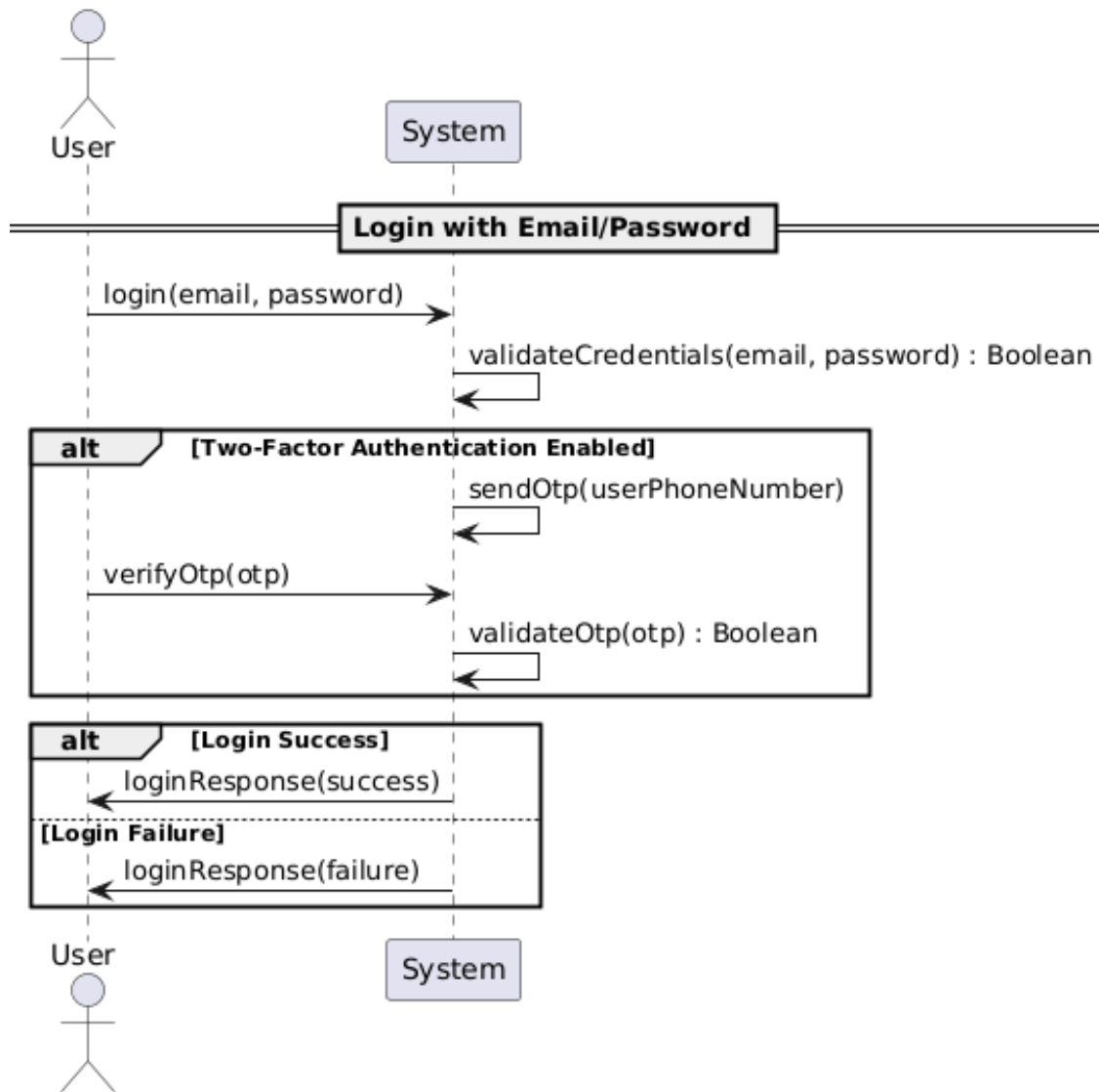


Figure 2.3: SSD for Login with Email/Password

2.2.3 SSD 3: Login with Google

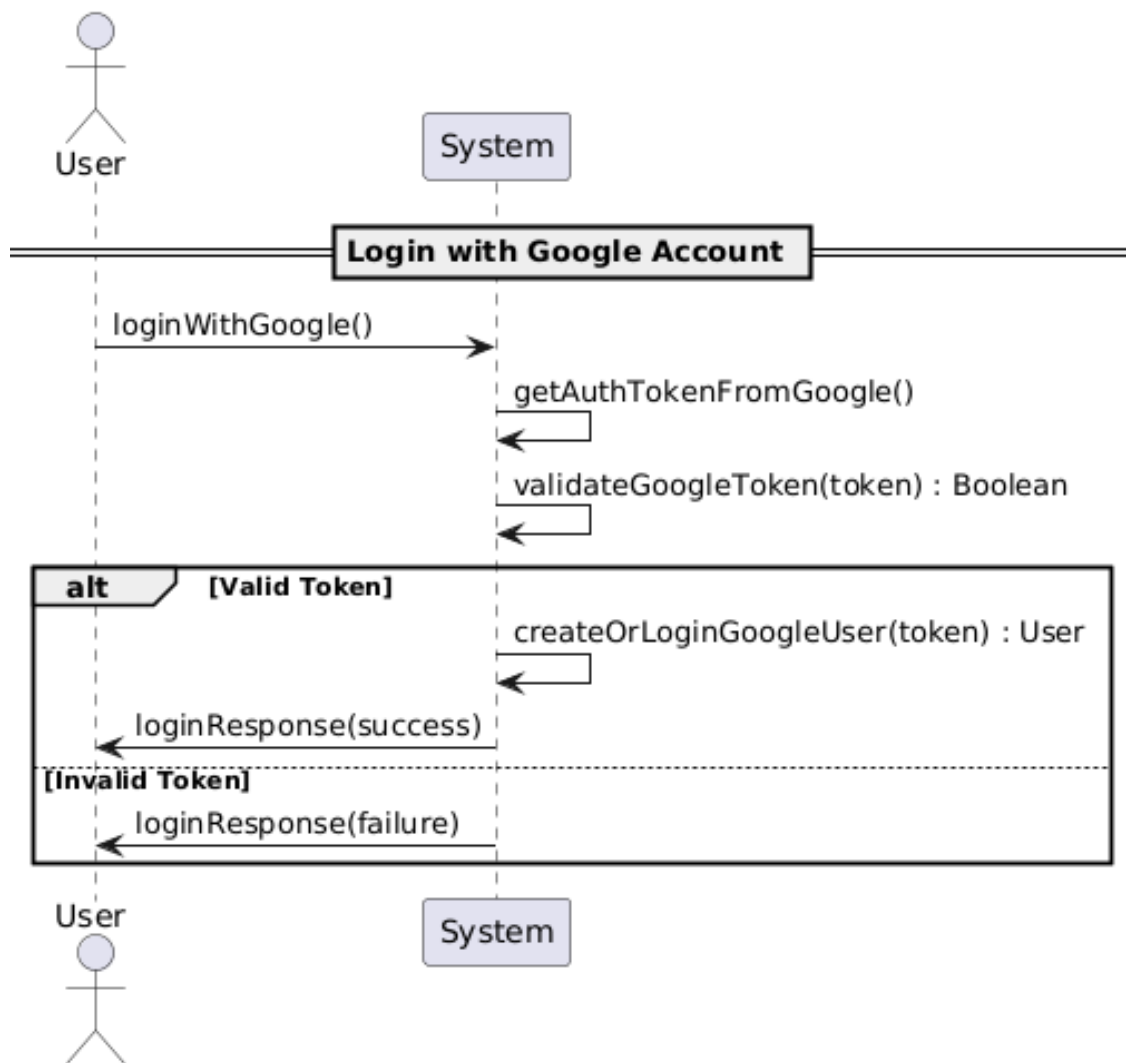


Figure 2.4: SSD for Login with Google

2.2.4 SSD 4: Password Reset

2.2.5 SSD 5: Automatic SMS and Notification Parsing for Transactions

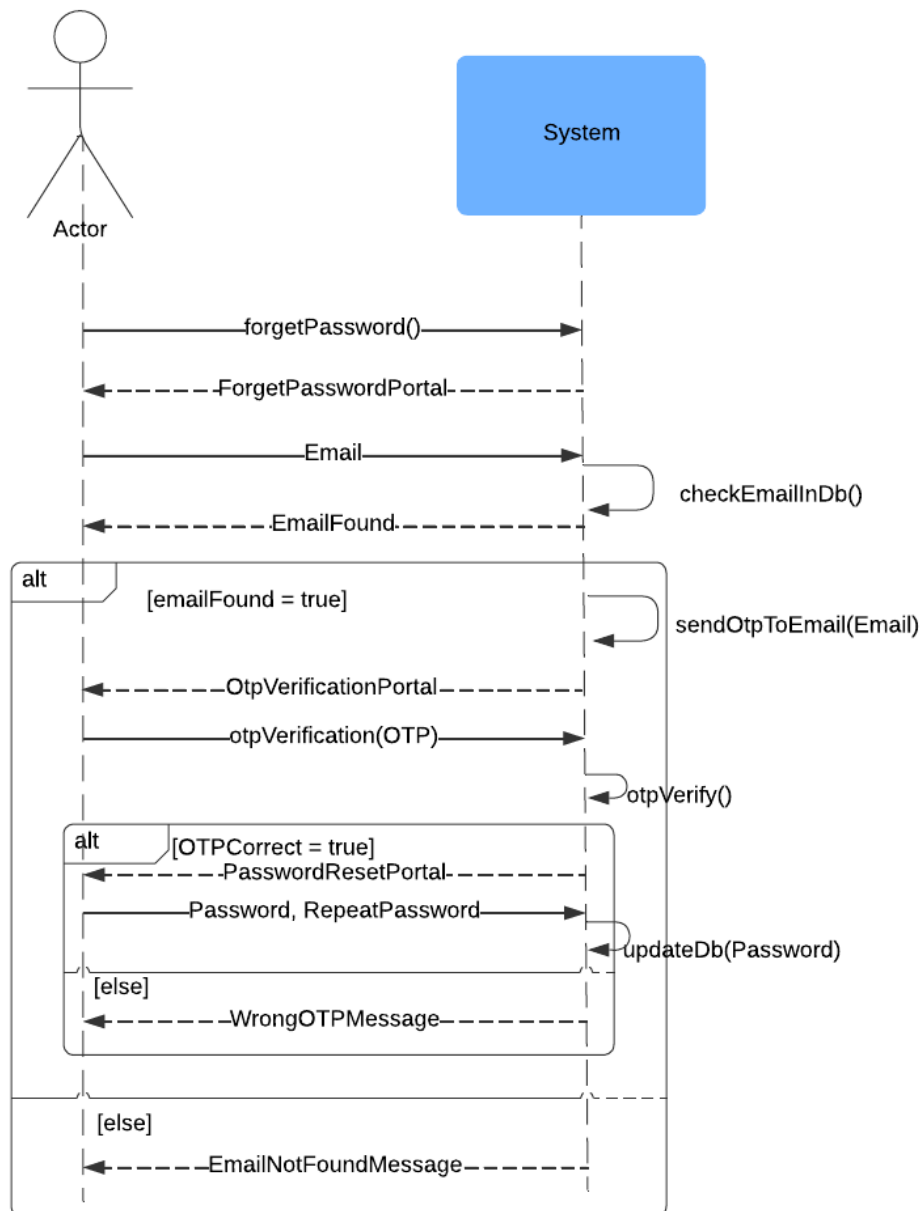


Figure 2.5: SSD for Password Reset

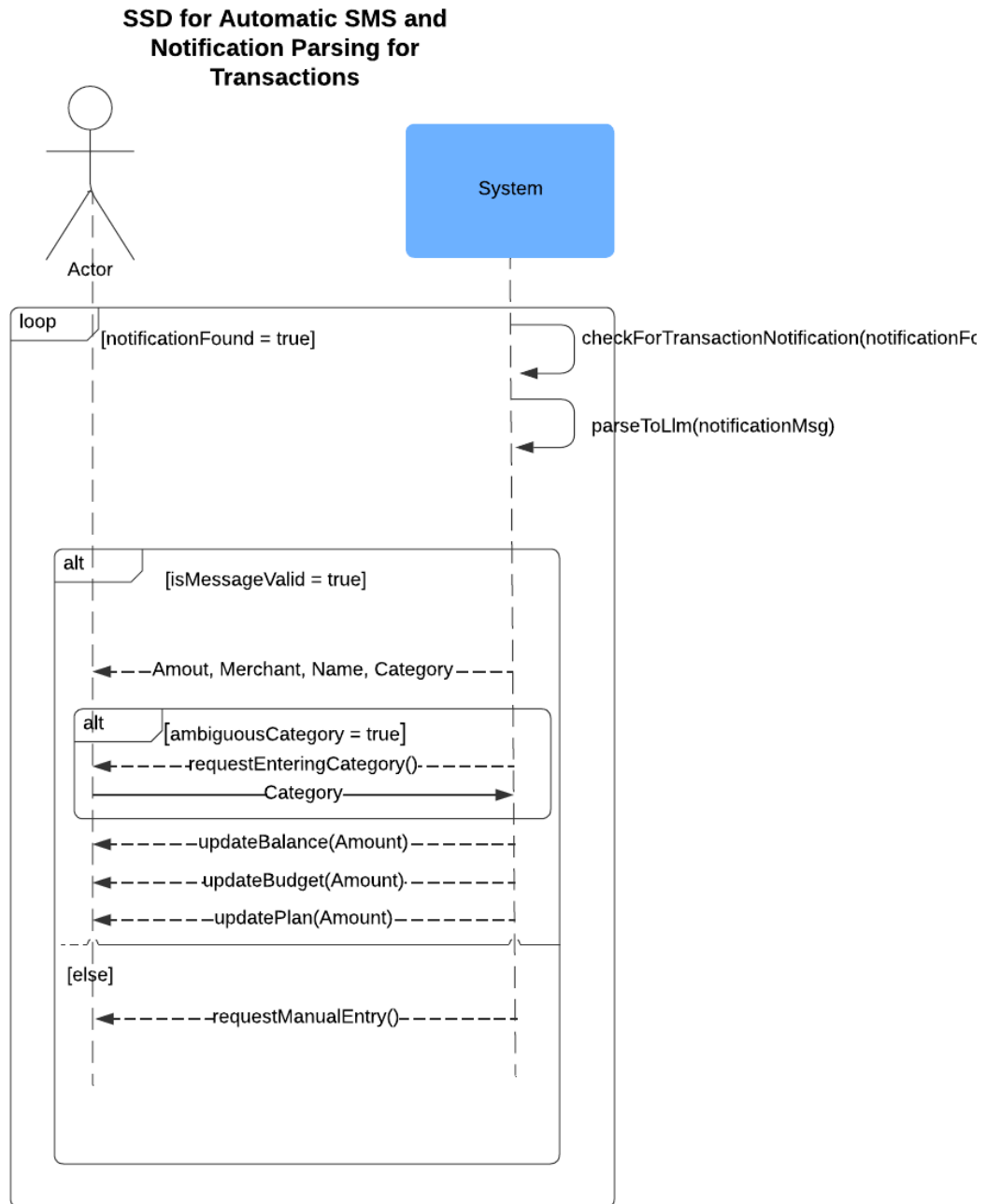


Figure 2.6: SSD for Automatic SMS and Notification Parsing for Transactions

2.2.6 SSD 6: Manual Transaction Entry

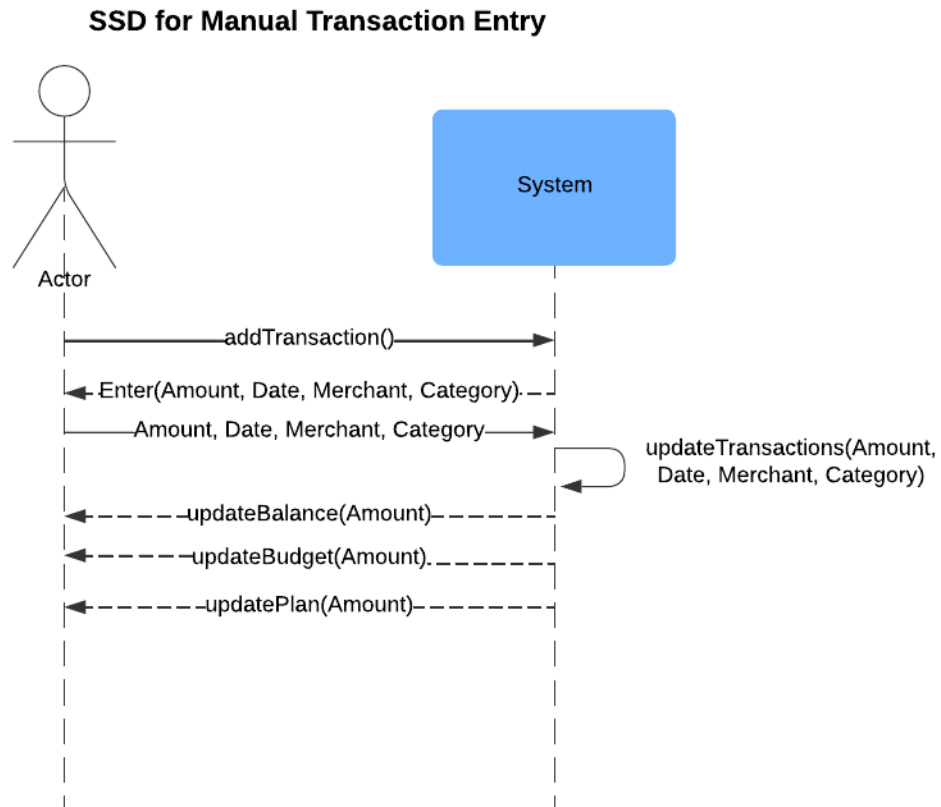


Figure 2.7: SSD for Manual Transaction Entry

2.2.7 SSD 7: View Transactions

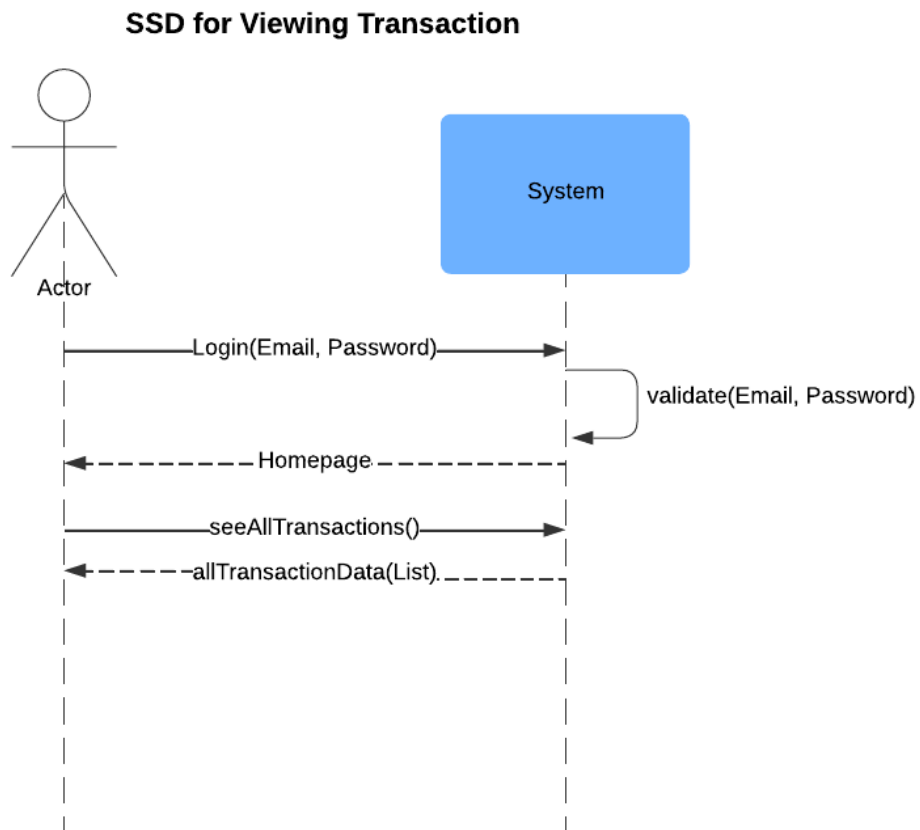


Figure 2.8: SSD for View Transactions

2.2.8 SSD 8: View Budgets

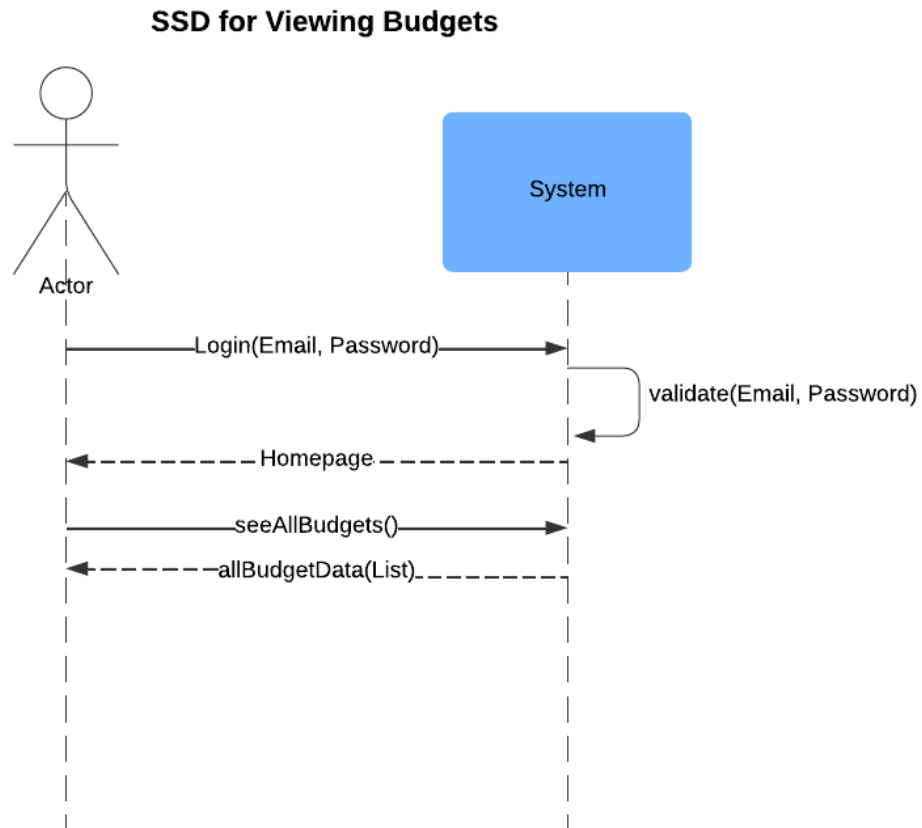


Figure 2.9: SSD for View Budgets

2.2.9 SSD 9: Create Budgets for Different Transaction Categories

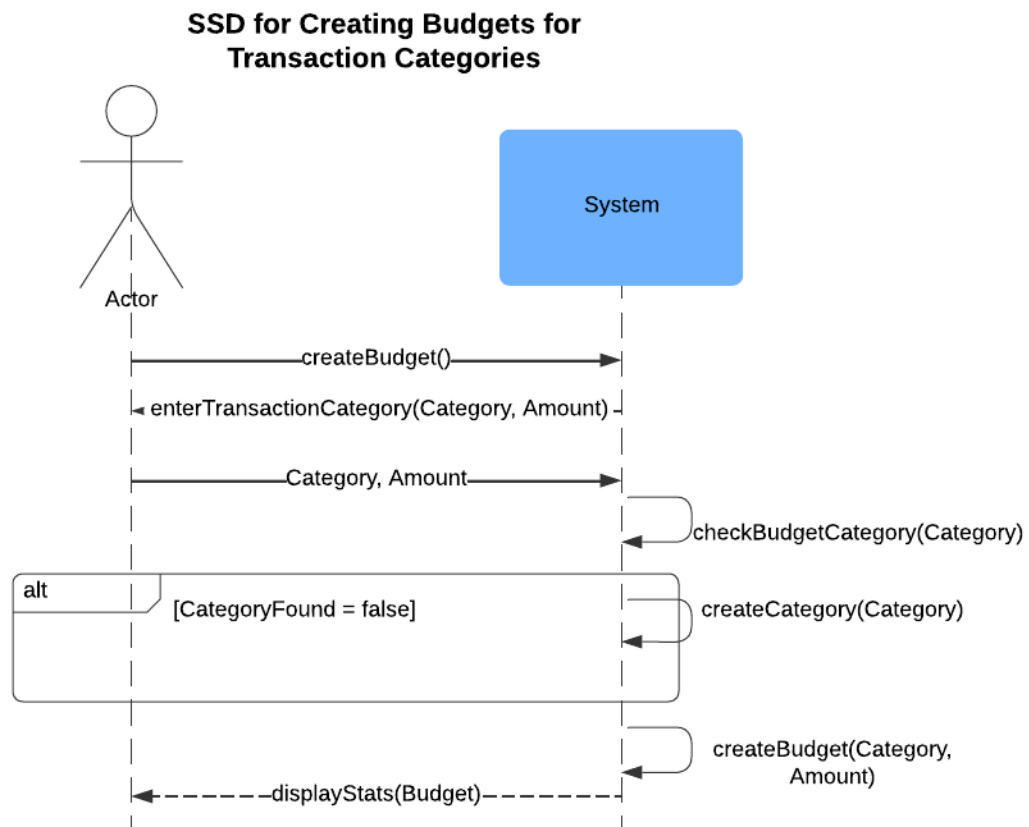


Figure 2.10: SSD for Create Budgets for Different Transaction Categories

2.2.10 SSD 10: Create New Categories

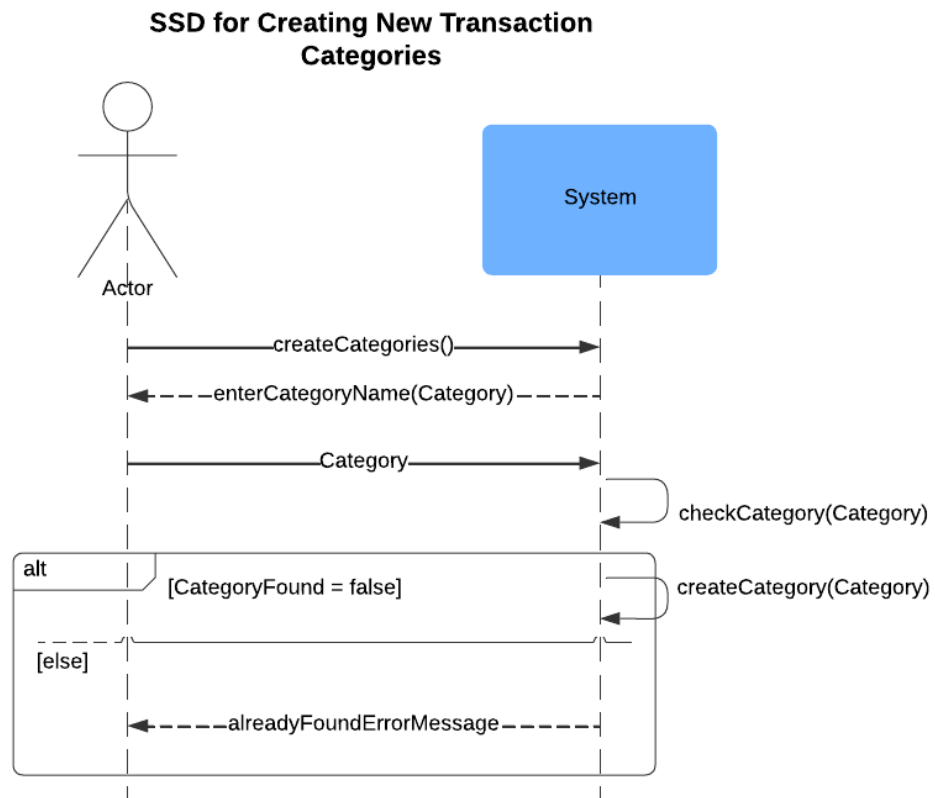


Figure 2.11: SSD for Create New Categories

2.3 System Sequence Diagrams

2.3.1 SD 1: User Registration

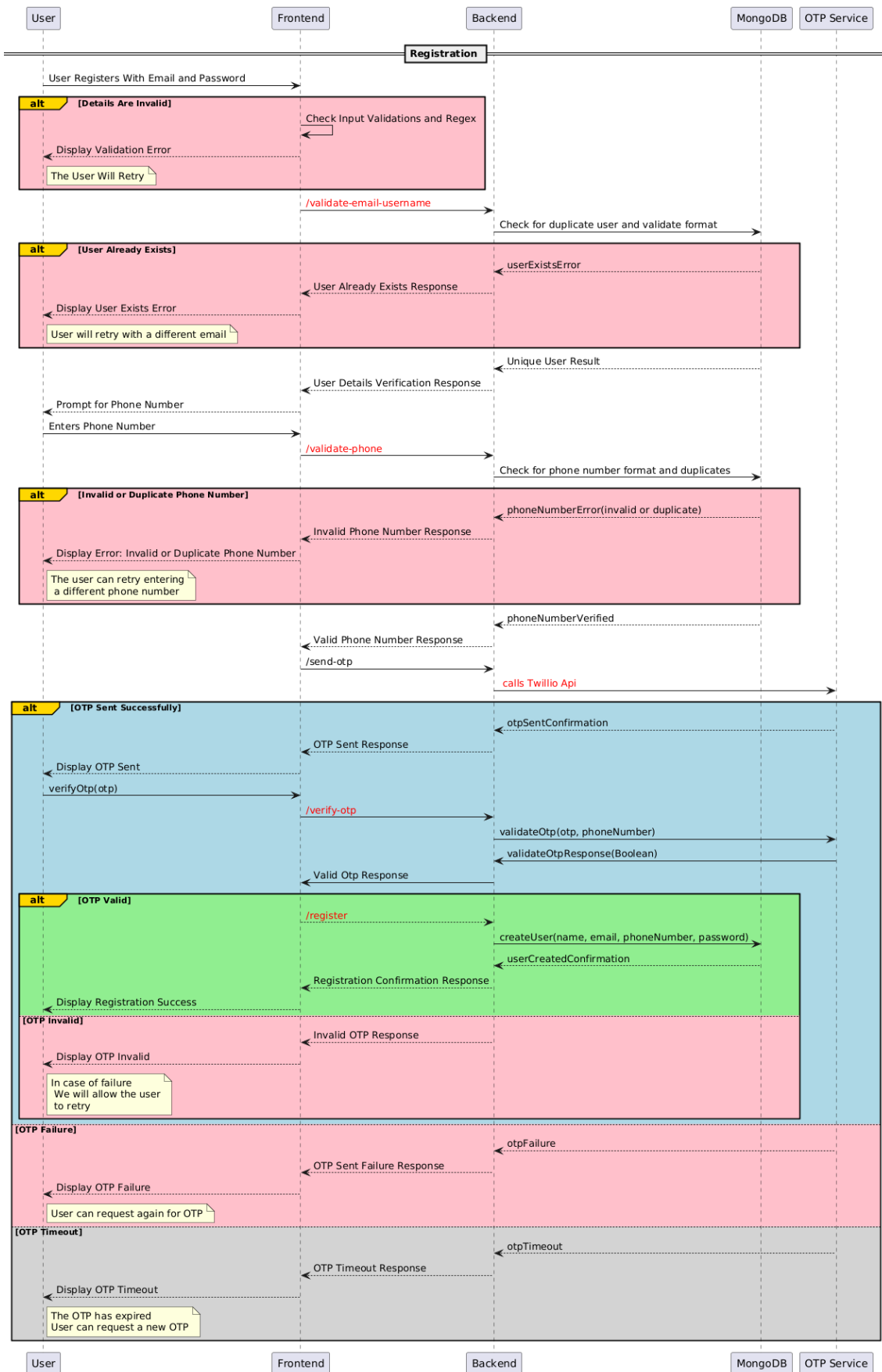


Figure 2.12: SD for User Registration

2.3.2 SD 2: Login with Email/Password

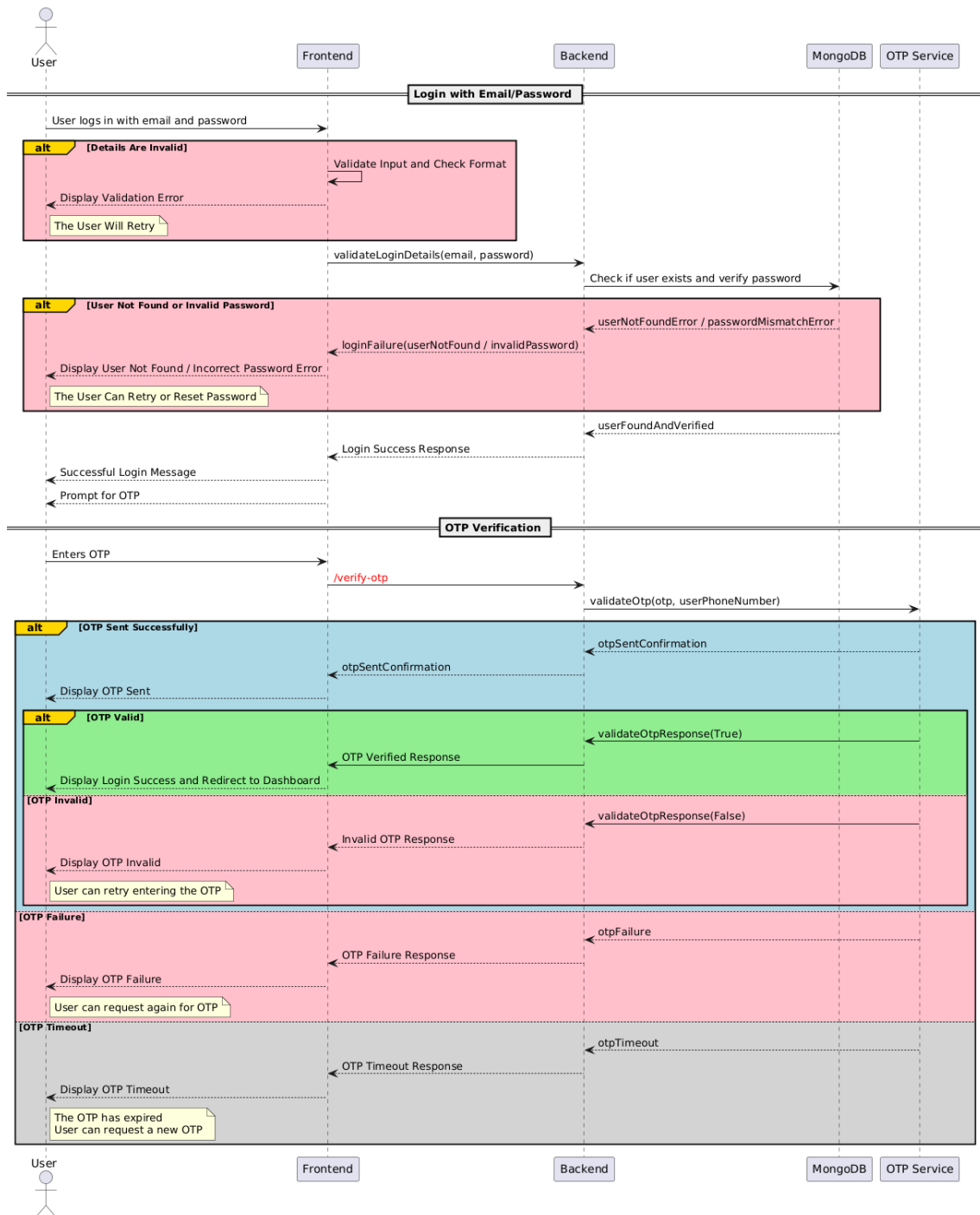


Figure 2.13: SD for Login

2.3.3 SD 3: Login with Google

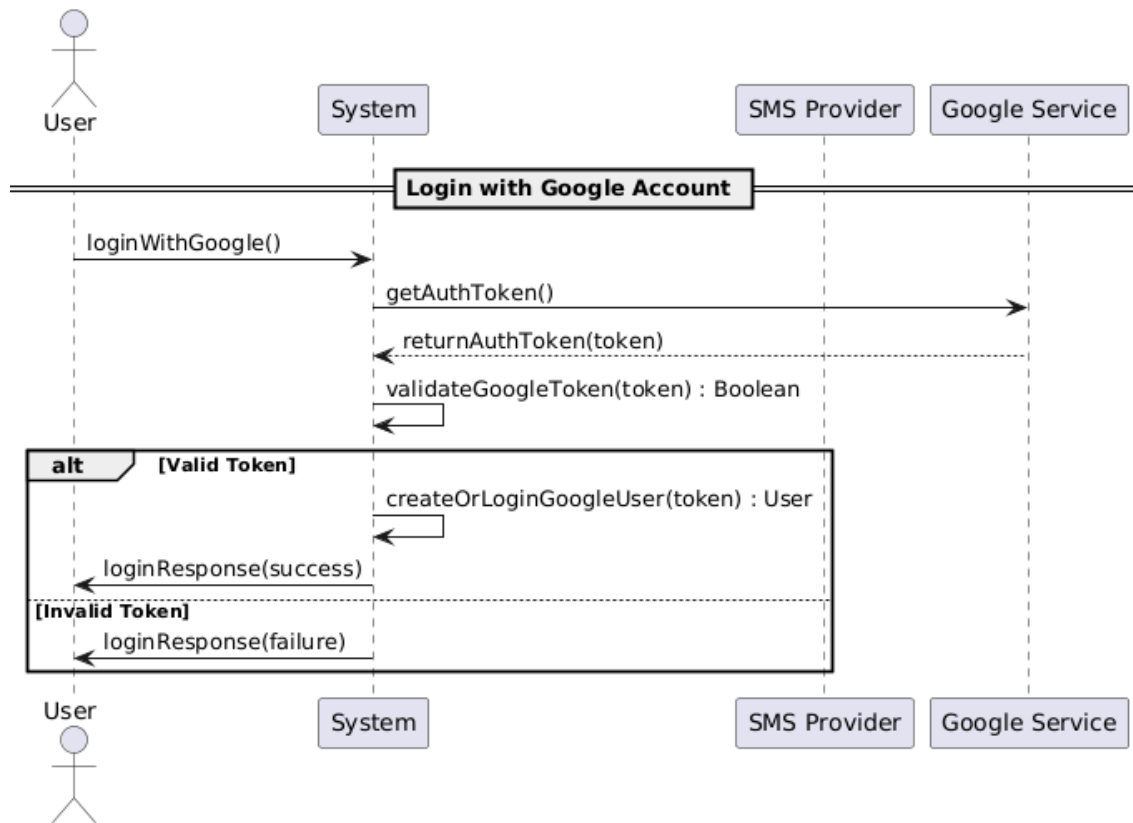


Figure 2.14: SD for Login with Google

2.3.4 SD 4: Password Reset

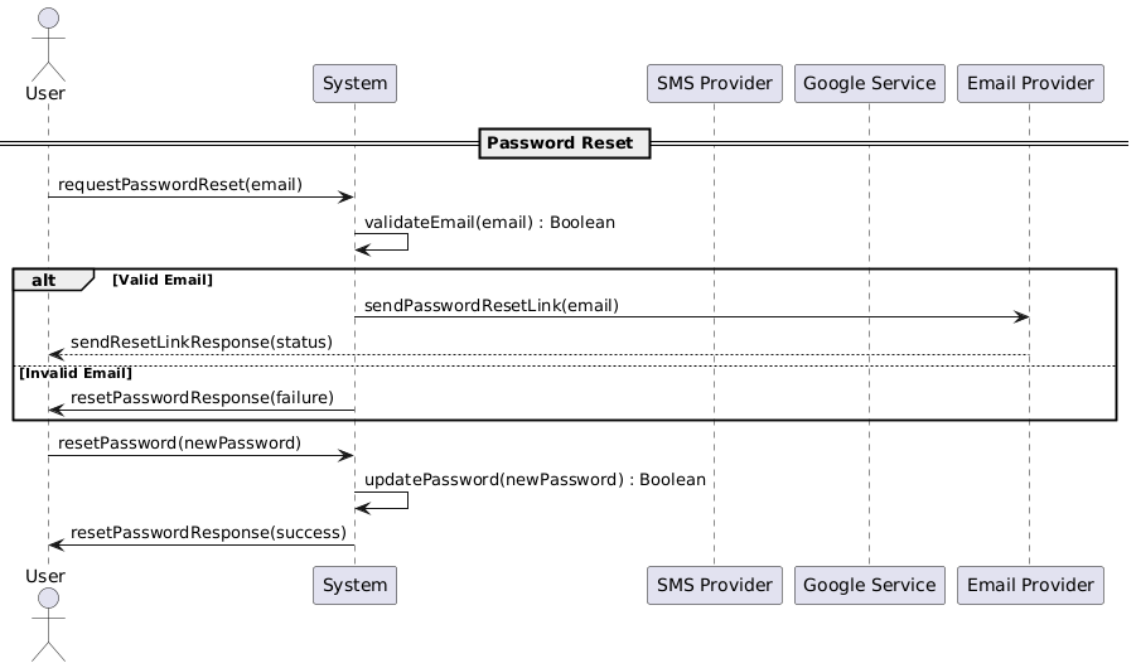


Figure 2.15: SD for Password Reset

2.3.5 SD 5: Automatic SMS and Notification Parsing for Transactions

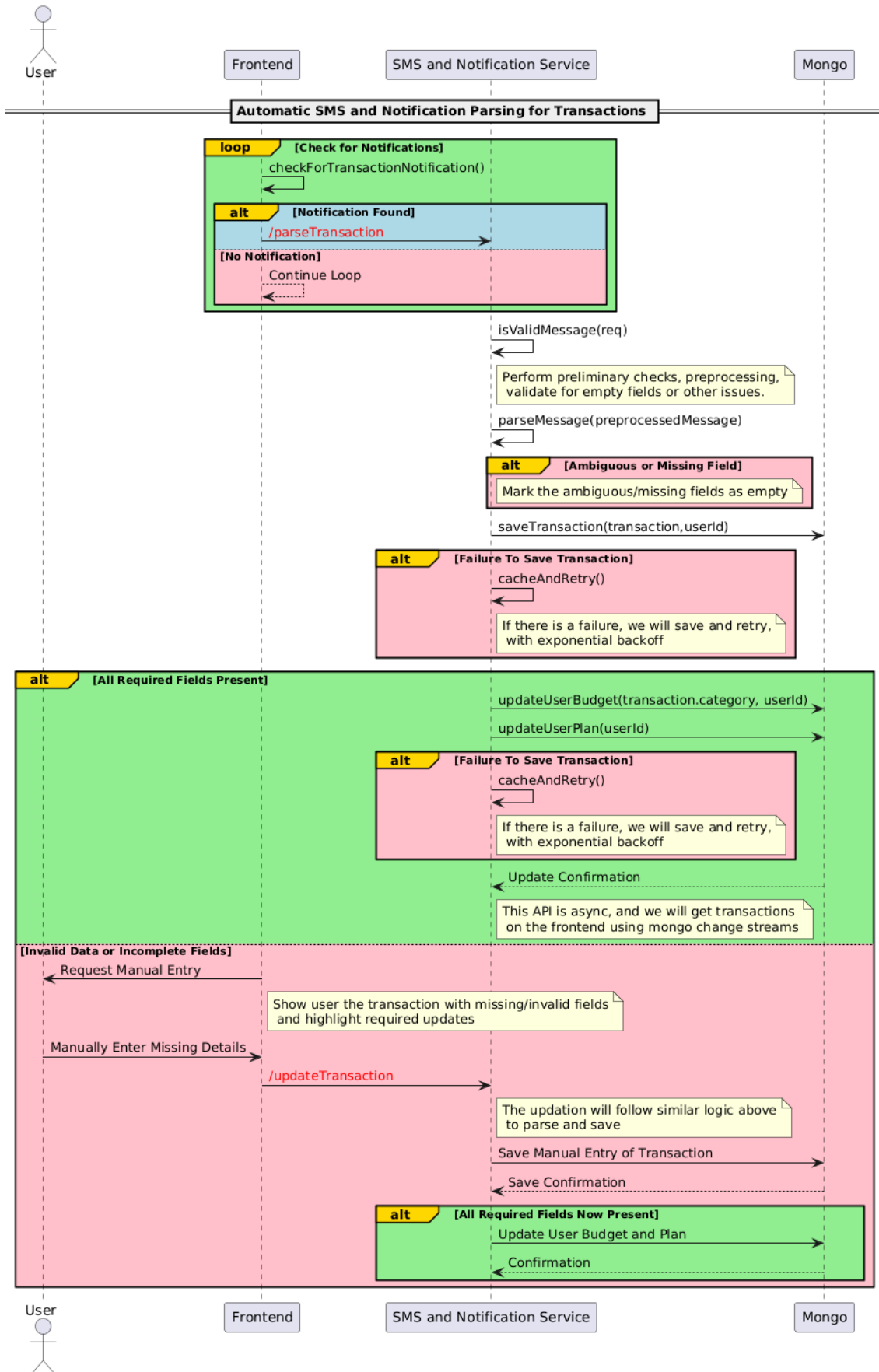


Figure 2.16: SD for Automatic SMS and Notification Parsing for Transactions

2.3.6 SD 6: Manual Transaction Entry



Figure 2.17: SD for Manual Transaction Entry

2.3.7 SD 7: View Transactions

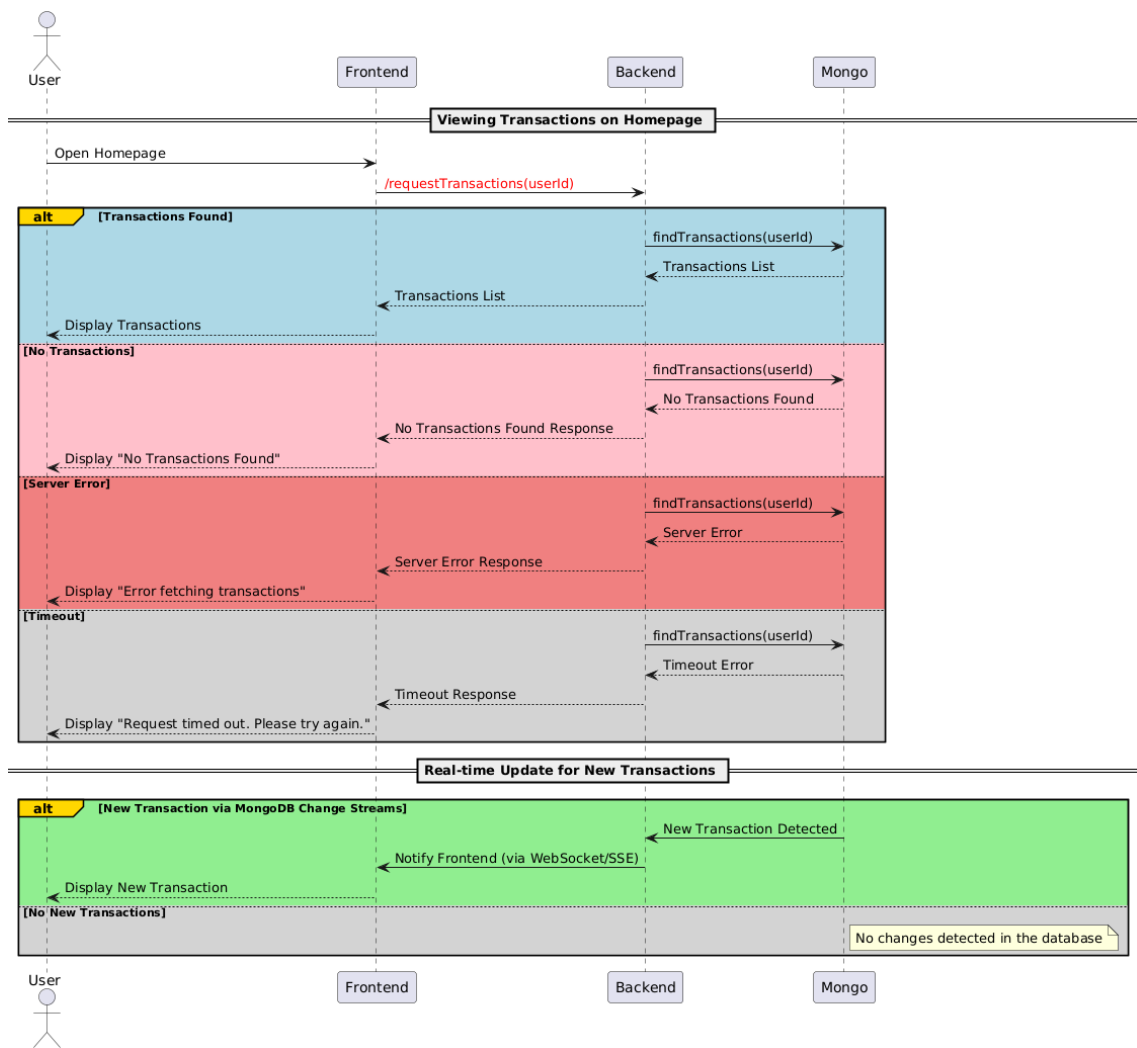


Figure 2.18: SD for View Transactions

2.3.8 SD 8: View Budgets

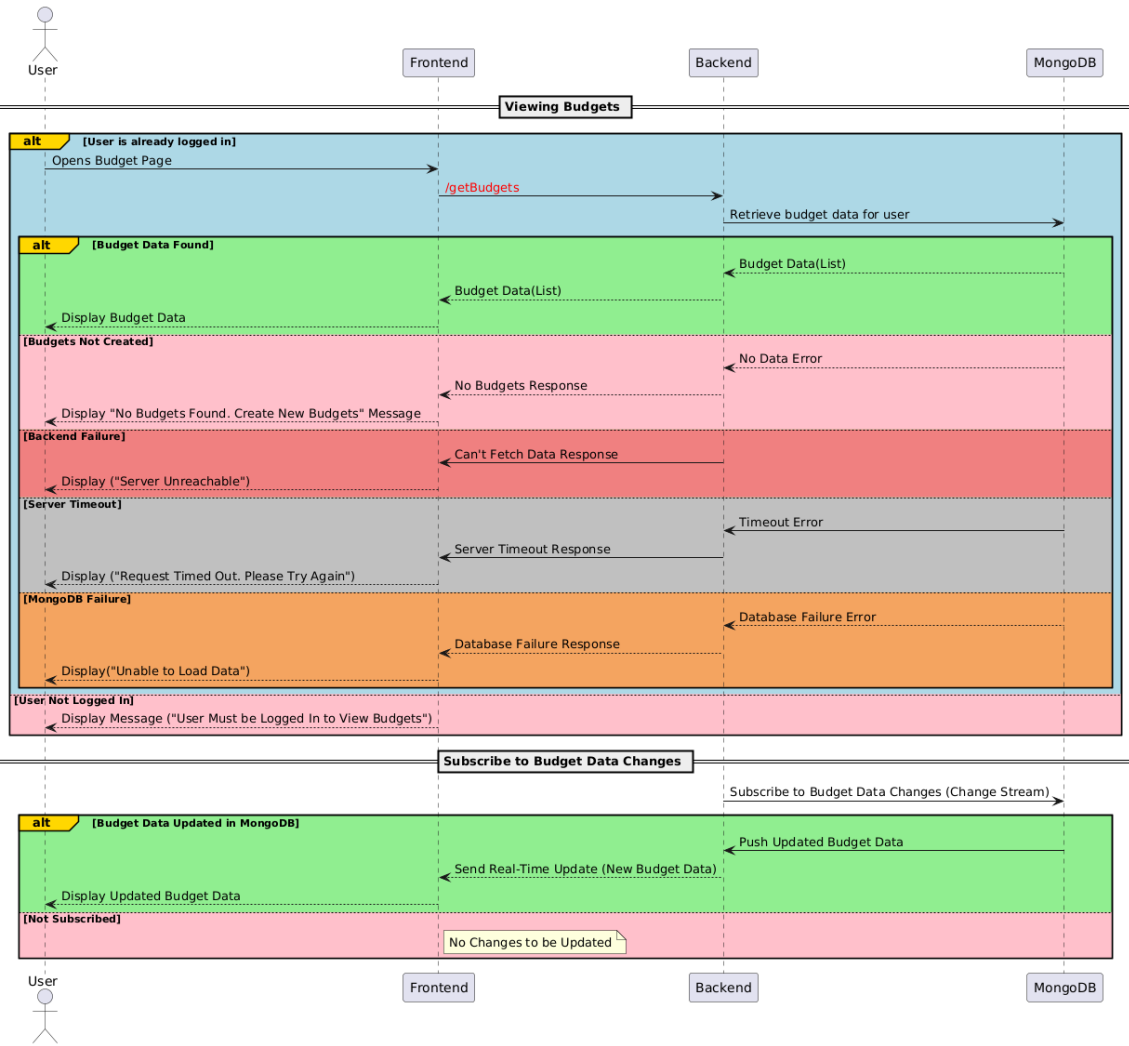


Figure 2.19: SD for View Budgets

2.3.9 SD 9: Create Budgets for Different Transaction Categories

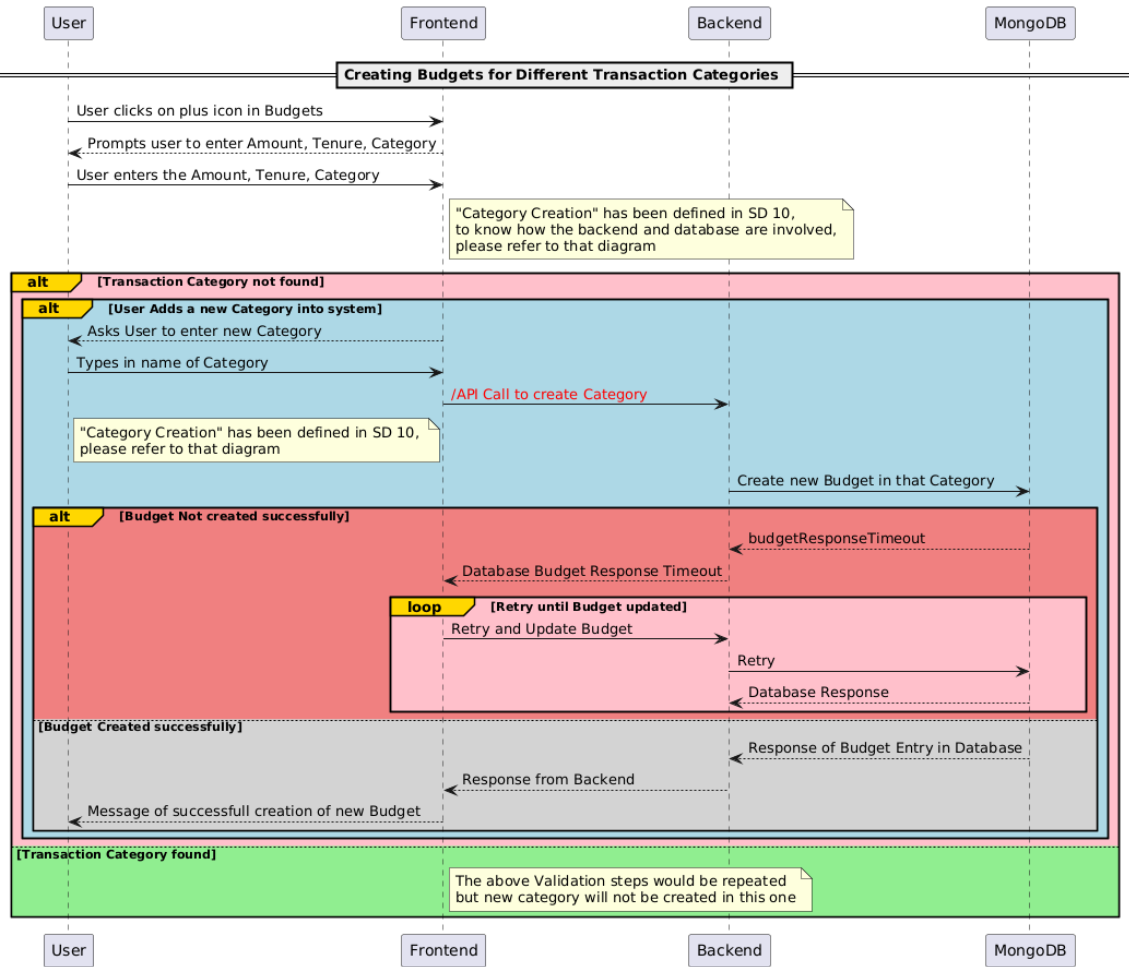


Figure 2.20: SD for Create Budgets for Different Transaction Categories

2.3.10 SD 10: Create New Categories

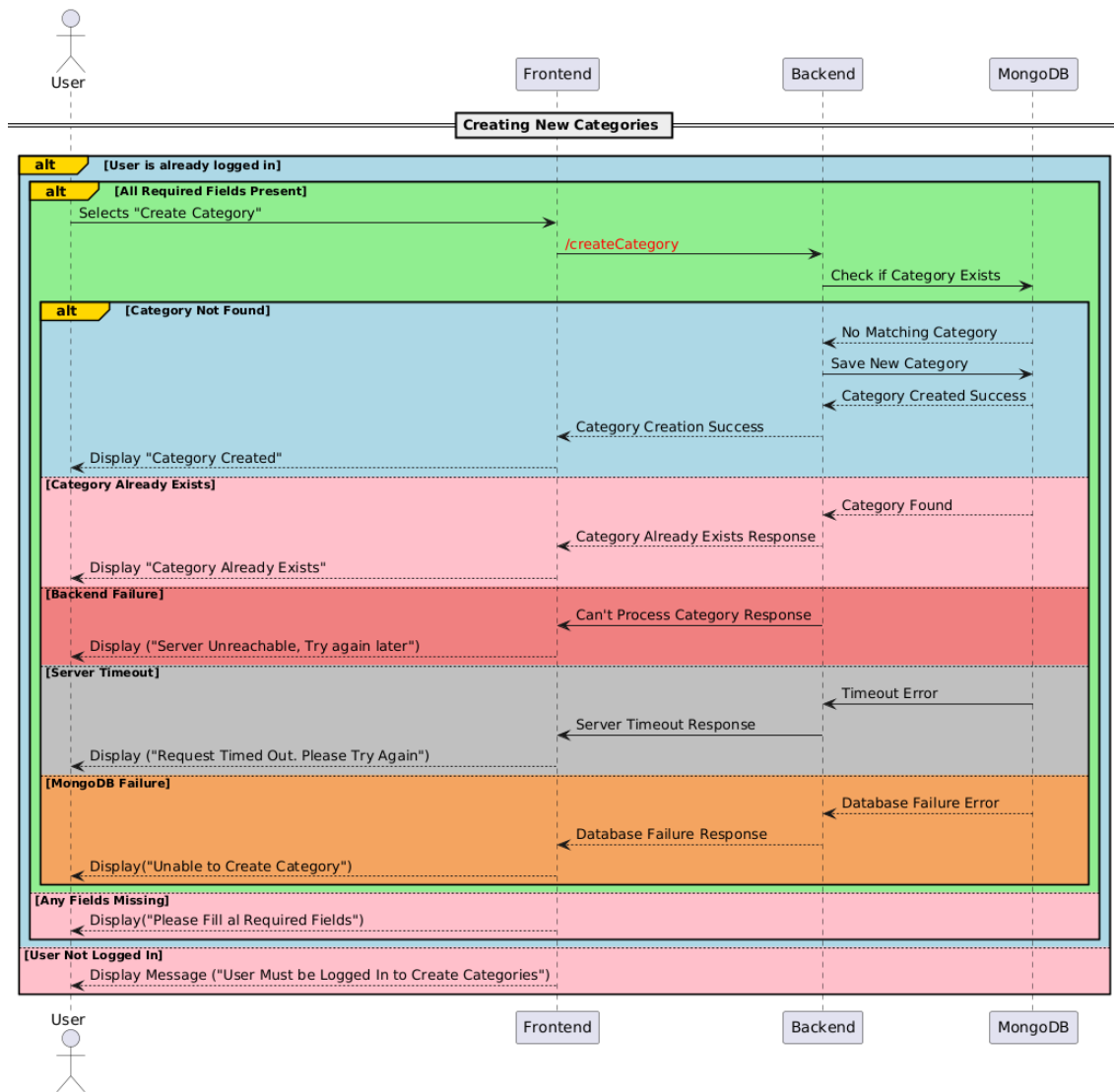


Figure 2.21: SD for Create New Categories

2.4 Functional Requirements

2.4.1 Module 1: SMS and Notification Reading For Automated Transactions

Following are the requirements for module 1:

- **FR 1) Login and Registration**
 - FR 1.1 The system shall allow new users to register by providing necessary details such as name, email, and password.
 - FR 1.2 The system shall send a verification SMS to the user whilst registering.

- FR 1.3 The system shall allow registered users to log in using their email and password.
- FR 1.4 The system shall also allow the user to register and sign in with their Google accounts.
- FR 1.5 The system shall provide users with an option to reset their password if they forget it.
- FR 1.6 The system shall support secure authentication mechanisms such as two-factor authentication.
- **FR 2) SMS and Notification Reading for Automatic Transaction Retrieval**
 - FR 2.1 The system shall automatically detect and read SMS messages containing the transaction details.
 - FR 2.2 The system shall automatically detect and read notifications containing the transaction details.
 - FR 2.3 The system shall extract relevant transaction information (amount, time, merchant name, and date) from the SMS and notification messages.
 - FR 2.4 The system shall categorize the extracted transactions based on predefined categories such as spendings, earnings, utilities, and bills.
 - FR 2.5 The system shall display a notification to the user after a successful transaction has been read and categorized.
 - FR2.6 The system shall allow users to manually verify and edit the transaction details.
 - FR2.7 The system shall update all interlinked information such as user plans when a new transaction is read
- **FR3) Ability to Manually Add Transactions**
 - FR3.1 The system shall provide an interface for users to manually add transaction details such as amount, date, time, merchant, category, and offline/online transaction
 - FR3.2 The system shall allow users to select from predefined categories when adding a transaction manually.
 - FR3.3 The system shall update all interlinked information such as user plans when a new transaction is entered manually
- **FR4) Ability to Categorize Transactions**
 - FR4.1 The system shall automatically categorize transactions based on keywords and patterns in the transaction message.

- FR4.2 The system shall allow users to manually reassign categories for their transactions.
- FR4.3 The system shall support the creation of custom categories by the user.
- FR4.4 The system shall enable users to view all transactions grouped by category.
- **FR5) Creating Budgets for Different Transaction Categories**
 - FR5.1 The system shall allow users to create custom budgets or predefined specific categories, specifying a budget and time
 - FR5.2 The system shall track spending against the budget in real-time and notify users when they are approaching or exceeding their budget.
 - FR5.3 The system shall provide visual representations of budget performance over time.
 - FR5.4 The system shall allow users to adjust or reset budgets at any time

2.4.2 Module 2: Uploading Bills and Receipts via Images

Following are the requirements for module 2:

- **FR6) Uploading Images and Screenshots for Automatic Transaction Processing**
 - FR6.1 The system shall allow the user to upload images of bills and screenshots of notifications to the system to automatically process them
 - FR6.2 The system shall allow the user to capture photos of receipt or bill containing transaction details directly within the app using the in-app camera.
 - FR6.3 The system shall allow the user to share screenshots directly from the Photos app into the budgeting app through a custom share extension.
 - FR6.4 The system shall display the extracted information to the user for confirmation or correction before logging the transaction.
- **FR7) Visualization of Categories and Spending**
 - FR7.1: The system shall allow users to create different financial scenarios based on variables such as income changes, estimated spendings etc. which will allow them to see the impact on their finances as well as their plans
 - FR 7.2 The system shall allow users to simulate the impact of potential tax law changes on their financial plans, helping them understand their future opportunities and finances

- FR 7.3 The system shall incorporate periodically updated inflation data to adjust to all financial scenarios
- FR 7.4 The system shall also allow users to input custom inflation rates manually as well
- 7.5: The system shall visualize these changes using graphs showing the impact on the user's finances and plans
- 7.6: The system shall allow users to compare multiple scenarios side by side

- **FR8) Sorting and Filtering Categories**

- FR8.1 The system shall allow users to sort transactions within a category by date, amount, and merchant name.
- FR8.2 The system shall provide filtering options to display transactions based on date/time ranges, categories, specific amounts, offline/online transactions or selected merchants.
- FR8.3 The system shall allow users to apply multiple filters simultaneously.
- FR8.4 The system shall support applying custom filters.

- **FR9) Notifications for Categorization, Budgets, and Transactions**

- FR9.1 The system shall notify users when they are about to exceed a budget.
- FR9.2 The system shall allow users to customize the frequency and timing of notifications.

2.4.3 Module 3: Planning and Forecasting

- **FR 10) Ability to Set Plans and Goals**

- FR 10.1 The system shall allow users to create financial plans and goals, specifying targets such as savings amounts.
- FR 10.2 The system shall allow users to create a custom plan/goal with options, along with predefined plans like car, mobile, Hajj/Umrah
- FR 10.3 The system shall provide users with progress tracking for each plan or goal, including percentage completion and time remaining.
- FR 10.4 The system shall allow users to modify or update their plans and goals based on changing circumstances.

- **FR 11) Scenario Planning and Visualization**

- FR 11.1 The system shall allow users to create different financial scenarios based on variables such as income changes, estimated spendings, etc., which will allow them to see the impact on their finances and plans.
- FR 11.2 The system shall allow users to simulate the impact of potential tax law changes on their financial plans, helping them understand future opportunities and finances.
- FR 11.3 The system shall incorporate periodically updated inflation data to adjust all financial scenarios.
- FR 11.4 The system shall also allow users to input custom inflation rates manually.
- FR 11.5 The system shall visualize these changes using graphs showing the impact on the user's finances and plans.
- FR 11.6 The system shall allow users to compare multiple scenarios side by side.

2.4.4 Module 4: Final Tweaks And Additional Features

Following are the requirements for module 2:

- **FR12) Stock Visualization**

- FR12.1: The system shall fetch and display historical stock data, including price trends, volume, and market performance.
- FR12.2 The system shall allow users to select specific stocks and visualize their performance

- **FR 13) Comparison of Different Insurance, Savings, and Retirement Plans**

- FR13.1: The system shall allow users to enter to select their own savings plan which will be factored into calculations
- FR13.2: The system shall provide a comparison between different insurance, savings, and retirement plans based on factors like coverage, premium, return rates, and benefits.
- FR13.3: The system shall provide visual representations to help users understand the differences between plans.

- **FR 14) Tax Templates**

- FR14.1: The system shall generate a tax template for users based on their income, transactions stored, and applicable tax laws

- FR14.2: The system shall provide users with a summary of their tax.
- FR14.3: The system shall allow users to export the tax template in a commonly used format, such as PDF and Word
- **FR 15) Reminders for Bill Payments, Partial Payments, Cash Withdrawals, and Other Manual Reminders**
 - FR15.1 The system shall allow users to set up reminders for bill payments, cash withdrawals, and other custom financial tasks and also send notifications.
 - FR15.2 The system shall allow users to specify the date, time, and frequency for each reminder.
 - FR15.3 The system shall enable reminders to be tied to specific transactions or categories.
 - FR15.4 The system shall notify users when a reminder is due, with an option to mark it as completed.

2.5 Non-Functional Requirements

This section specifies nonfunctional requirements.

- **NFR 1) Login and Registration**
 - NFR 1.1 The registration process shall be completed within 5 seconds after the user submits their details.
 - NFR 1.2 The system shall ensure a secure login process with response times of less than 3 seconds.
 - NFR 1.3 The system shall comply with security best practices to protect user credentials.
- **NFR 2) SMS and Notification Reading for Automatic Transaction Retrieval**
 - NFR 2.1 The system shall retrieve and process SMS and notification transactions within a minute of receiving the SMS.
 - NFR 2.2 The system shall be capable of parsing transaction details from at least 95% of all SMS and notifications.
 - NFR 2.3 The system shall ensure the privacy and security of SMS and notification data by encrypting transaction details both at rest and in transit.
 - NFR 2.4 The system shall attempt to differentiate between online and in-person transactions wherever possible, otherwise, it will allow the user to add this manually.

- **NFR 3) Ability to Manually Add Transactions**

- NFR 3.1 The system shall save manually added transactions within 2 seconds of user submission.
- NFR 3.2 The system's interface for adding manual transactions shall be responsive and optimized for various screen sizes, not including tablets and tvs.
- NFR 3.3 The system shall provide confirmation feedback to the user upon successful addition of a transaction.
- NFR 3.4 The system shall maintain a history of all manually added transactions for up to 1 year on the primary database, with older data maintained on a secondary database.
- NFR 3.5 The system shall validate the transaction data entered by the user to ensure all mandatory fields are completed before saving.

- **NFR 4) Ability to Categorize Transactions**

- NFR 4.1 The system shall ensure the categorization process is completed within 10 seconds for each transaction.
- NFR 4.2 The system shall maintain an accuracy rate of 90% or higher in automatic transaction categorization.
- NFR 4.3 The system shall synchronize user-defined categories across all user devices in real-time.

- **NFR 5) Creating Budgets for Different Transaction Categories**

- NFR5.1 The system shall update budget tracking data in real-time, reflecting changes in transaction categorization within 5 seconds.
- NFR5.2 The system shall display the budget progress within 5 seconds of accessing the budget screen.
- NFR5.3 The system shall support the creation and management of up to 4 active budgets per user without performance issues.
- NFR5.4 The system shall maintain budget history and performance data for up to 1 year on the primary database, with older data stored on the secondary database, allowing users to review past budgets.
- NFR5.5 The system shall give the option to create a budget from already created custom categories when the user selects the option for a custom budget.

- **NFR 6) Image Upload For Adding Transaction**

- NFR6.1 The system shall achieve an accuracy rate of at least 90% for extracting transaction details from screenshots and images

- NFR6.2 The in-app camera interface shall be intuitive and user-friendly, requiring no more than 3 taps to capture and process a photo.
- NFR6.3 The system shall process and display extracted data within 10 seconds of a screenshot being uploaded
- NFR6.4 The system shall allow users to easily retake a photo if the original capture was unclear or misaligned, with a maximum of 1 tap to retry.
- NFR6.5 The system shall provide an onboarding guide for the user.
- NFR6.6 The system shall not retain any unnecessary personal data from the image as well as the image itself once the transaction has been logged.

- **NFR7) Visualization of Categories and Spending**

- FR7.1 The system shall provide graphical representations (e.g., pie charts, bar charts, line graphs) of spending across different categories (e.g., groceries, utilities, entertainment).
- FR7.2 The system shall allow users to filter and sort visualizations by time period (e.g., daily, weekly, monthly, yearly).
- FR7.3 The system shall allow users to focus on specific categories to view detailed transactions contributing to the visualized data.

- **FR8) Sorting and Filtering Categories**

- NFR8.1 The system shall perform sorting and filtering operations within 10 seconds, regardless of the number of transactions.
- NFR8.2 The system shall handle sorting and filtering for up to 1000 transactions without affecting performance.
- NFR8.3 The system shall show the three most recently used filters
- NFR8.4 The system's sorting and filtering features shall be available offline and synchronize changes once the device is back online.

- **FR9) Notifications for Categorization, Budgets, and Transactions**

- NFR9.1 The system shall attempt to deliver notifications in real-time within a maximum of 30 seconds, but does not guarantee successful delivery of notifications
- NFR9.2 The system shall support push notifications on both iOS and Android platforms.
- NFR9.3 The system shall ensure that notifications are non-intrusive and do not overwhelm the user, with an option to aggregate related notifications.

- **NFR 10) Ability to Set Plans and Goals**

- NFR10.1 The system shall store and manage plans and goals efficiently, ensuring data consistency and integrity.
- NFR10.2 The progress tracking for plans and goals shall be updated in real-time within 15 seconds after new data is entered into the system.
- NFR10.3 The goal-setting interface shall be intuitive and user-friendly, enabling users to set up plans and goals with minimal effort.

- **NFR 11) Scenario Planning and Visualization**

- NFR11.1 The system shall incorporate a machine learning model that will take in various parameters and generate scenarios.
- NFR11.2 The model engine shall process user input and generate forecasts within 30 seconds.
- NFR11.3 The system shall ensure that all graphs and visualizations will have consistent display on both iOS and Android.
- NFR11.4 The system shall ensure that proper disclaimers are shown to the user before visualizing a plan.
- NFR11.5 The system shall maintain user privacy and security by ensuring that all data used in predictive models is anonymized and encrypted.
- NFR11.6 The system shall be transparent, ensuring that users can see and understand the data and assumptions used in the models.

- **NFR 12) Stock visualization**

- NFR12.1: The system shall retrieve and update stock data with a latency of no more than 10 seconds.
- NFR12.2: The system shall be capable of visualizing stock data for at least 5 different stocks
- NFR12.3: The system shall ensure that all stock data is up-to-date and synchronized within 24 hours of the data being published

- **NFR 13) Comparison of Different Insurance, Savings and Retirement Plans**

- NFR13.1: The system shall display comparisons of insurance, savings, and retirement plans within 5 seconds of the user's request.
- NFR13.2: The comparison data shall be accurate, reflecting updates processed within 24 hours of changes.
- NFR13.3: The system shall support the comparison of at least 3 different plans simultaneously without affecting performance or accuracy.

- NFR13.4: The system shall ensure that references and links are provided for the user
- NFR13.5: The system shall ensure that disclaimers are provided in regards to investing in these plans to make the user more aware and to take blame away from the system
- **NFR 14) Tax Template**
 - NFR14.1: The tax template generation process shall be completed within 15 seconds
 - NFR14.2: The system shall allow exporting tax templates in a secure format, ensuring that the file is encrypted and accessible only to the authorized user
 - FR14.3: The system shall provide a disclaimer to the user and the terms and conditions before generating a tax template
- **NFR 15 Reminders for Bill Payments, Partial Payments, Cash Withdrawals, and Other Manual Reminders**
 - NFR15.1 Reminders shall be synced across devices when the user is logged into the app on multiple platforms.

2.5.1 Reliability

Requirements about how often the software fails. The measurement is often expressed in MTBF (mean time between failures). The definition of a failure must be clear. Also, don't confuse reliability with availability which is quite a different kind of requirement. Be sure to specify the consequences of software failure, how to protect from failure, a strategy for error detection, and a strategy for correction.

Failure shall be defined as server crashing and no responses from the backend. Failure could be due to pod issues, cluster problems, database availability.

- NFR 16.1 The system shall have backups in multiple zones. This will be ensured later on into development
- NFR 16.2 The system shall protect the app from failure by utilizing backups and best available cloud architectures to support large traffic.
- NFR 16.3 The system shall have disaster recovery and high availability servers in case of backups
- NFR 16.4 The system shall detect failures by using liveness probe provided by the kubelet in kubernetes.

- NFR 16.5 The system shall use backup databases, master node architecture, and sharding to allow recovery in case of failures.
- NFR 16.6 In case of correction, we can rollback to a previous deployment using kubernetes control plane
- NFR 16.8 The system shall achieve detection of failures by a custom written exec-liveness.yaml file in the kubernetes project.

2.5.2 Usability

Usability requirements have been specified above with the requirements

2.5.3 Performance

We have already mentioned performance non functional requirements above with each NFR

2.5.4 Security

In addition to our previous security requirements, we are also adding other security requirements such as:

- NFR 17.1 The system shall utilize MD5 or SHA algorithm for encrypting transaction information.
- NFR 17.2 User will be required to enter a strong password with at least 8 characters with a upper case letter, and a number or a special character.
- NFR 17.3 The system shall utilize AES algorithm for encrypting sensitive user information like the password.
- NFR 17.4 The system shall ensure user's data privacy and safety by conducting external audits of our application's security and data handling practices.
- NFR 17.5 The system shall use the principle of limiting to only allow users access to as much data they are required

2.6 Domain Model

Create a representation of the domain model for your project.

Apna Budget: Domain Model

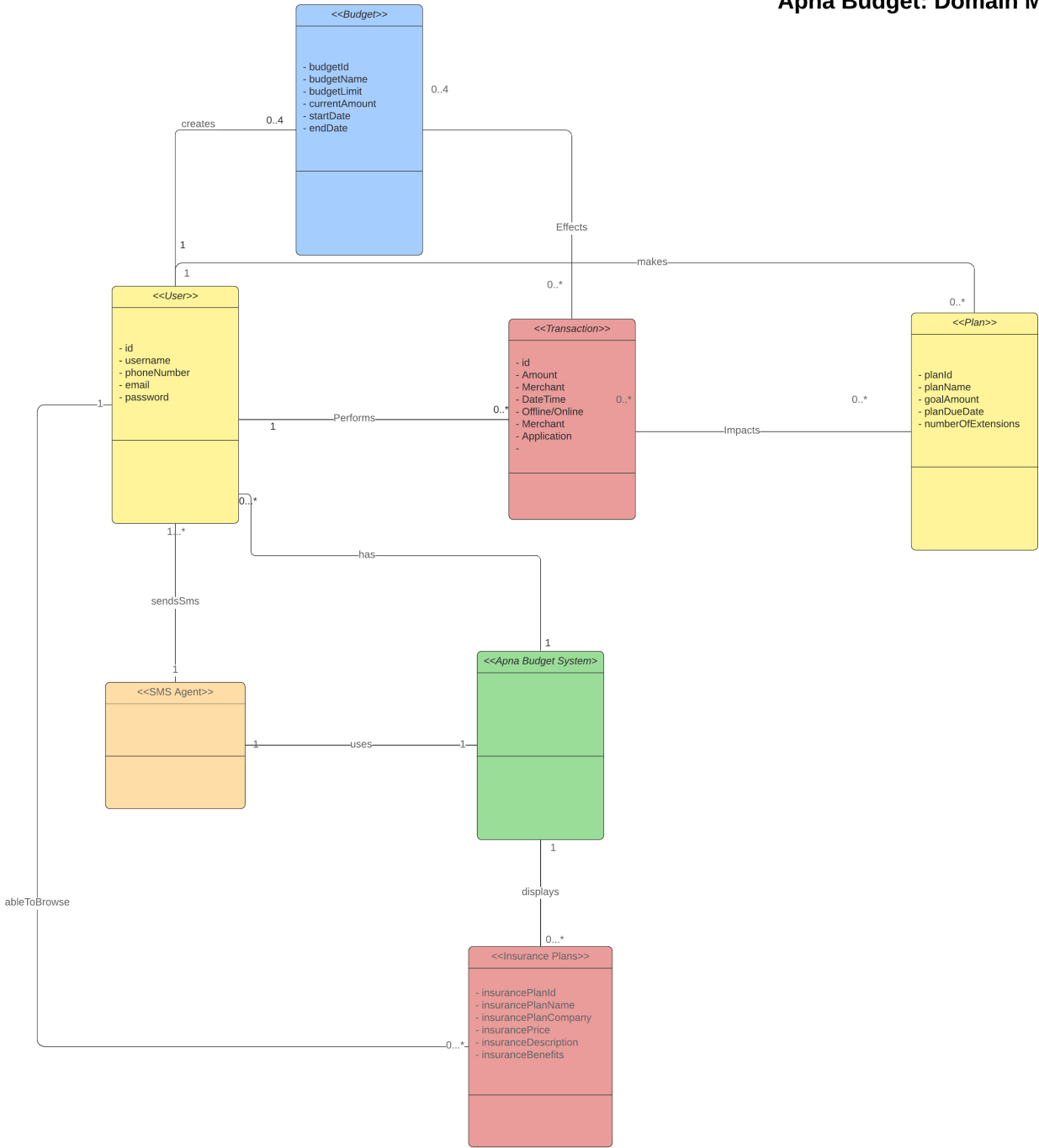


Figure 2.22: Domain Model of Apna Budget

Chapter 3

System Overview

Apna Budget's mobile app is built on Flutter, offering a cross-platform experience across Android and iOS with offline support using SQLite. Firebase handles remote updates, ensuring a dynamic user experience without the need for constant app updates. It's all about efficiency and user convenience. The backend uses Kubernetes to manage multiple microservices, including user management, SMS processing, and budgeting. Nginx directs traffic through an API Gateway, while MongoDB and Redis handle data storage and caching. With Kafka facilitating message processing, the system is scalable and reliable. Security is key, so Twilio provides OTP authentication, keeping user data safe. We rely on the ELK stack for logging and monitoring, giving us real-time insights and helping maintain system integrity and performance at all times.

3.1 Architectural Design

The Apna Budget system is divided into key modules: the frontend, backend services, and the data layer, ensuring each part has a focused responsibility. The frontend, built on Flutter, communicates with the backend via an API Gateway, making the user experience seamless across Android and iOS. Backend services, including budgeting, SMS processing, and user management, are hosted in a Kubernetes cluster, allowing for scalability and isolated development. MongoDB stores persistent data, while Redis acts as a cache to enhance performance. Kafka enables efficient communication between microservices, keeping everything in sync. This modular design not only simplifies scaling but also improves maintainability, ensuring each module can evolve independently without affecting others.

- Architecture Diagram

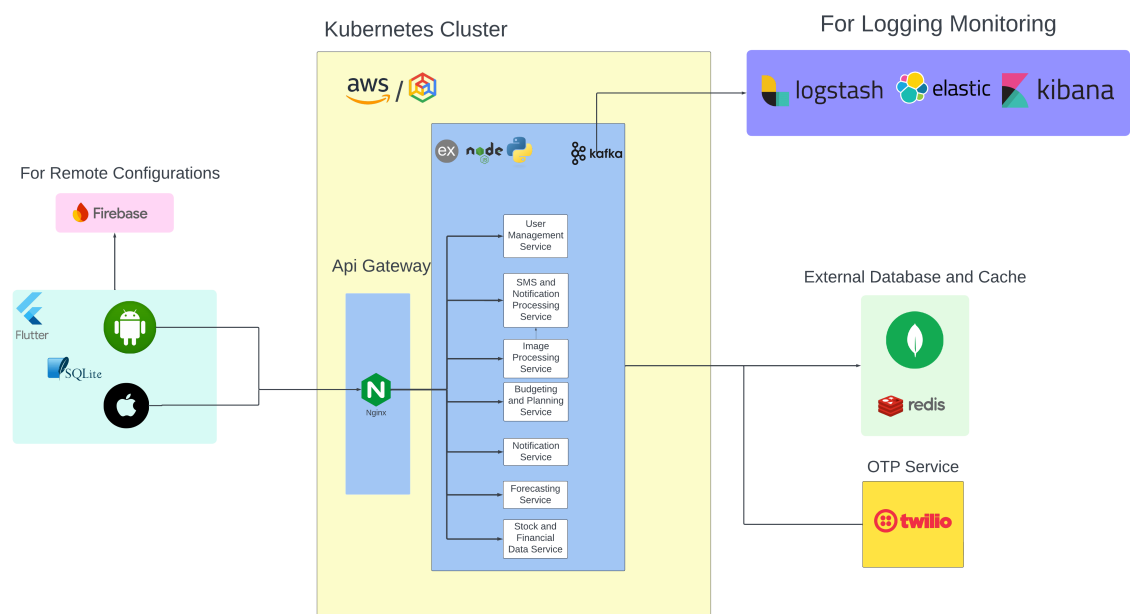


Figure 3.1: Architecture Diagram of Apna Budget

- Component Diagram

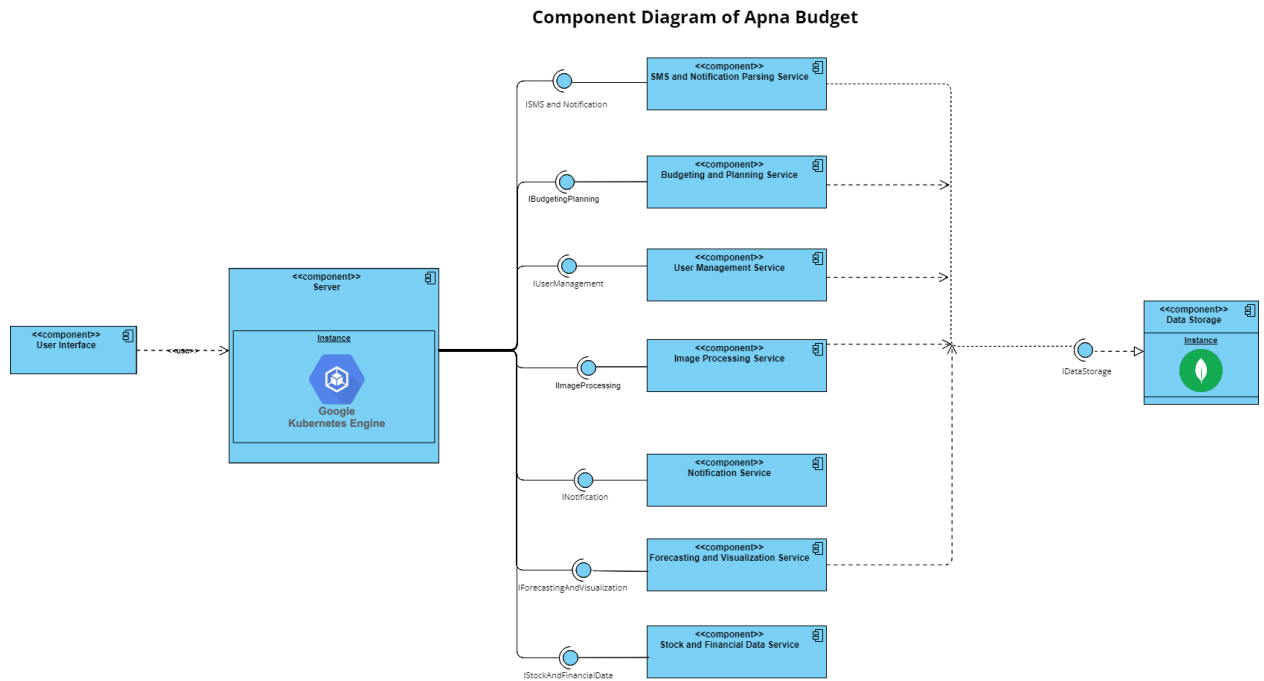


Figure 3.2: Component Diagram of Apna Budget

3.2 Design Models

The design models for this application encompass both an Activity Diagram and a Class Diagram, which together provide a comprehensive view of user interactions and system architecture. The Activity Diagram represents a roadmap for user actions, effectively outlining the flow from logging in to managing budgets, while helping in understanding of user experiences and pathways through the app. The Class Diagram describes the backend structure, illustrating the relationships among various components, such as `AuthService`, `BudgetController`, and `TransactionService`, showcasing how they work together to manage user data and functionalities. Together, these models ensure that the development align with users needs, clarifying the application's functionality while making a good system design.

Design Models for Apna Budget

The data models for Apna Budget include:

- Activity Diagram

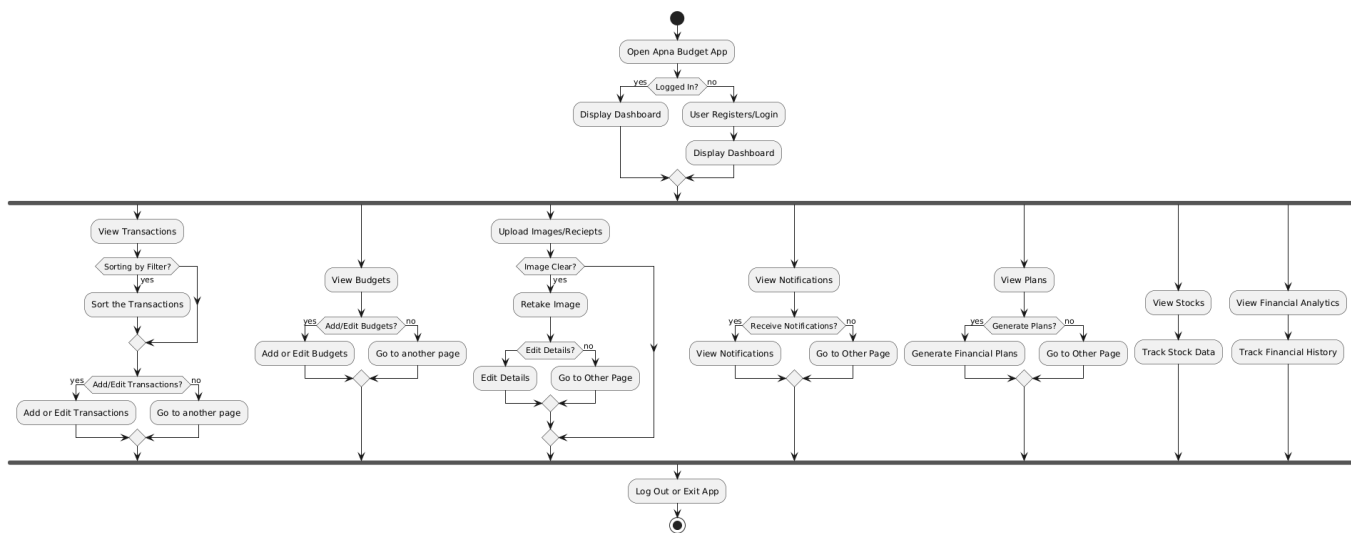


Figure 3.3: Activity Diagram of Apna Budget

• Package Diagram

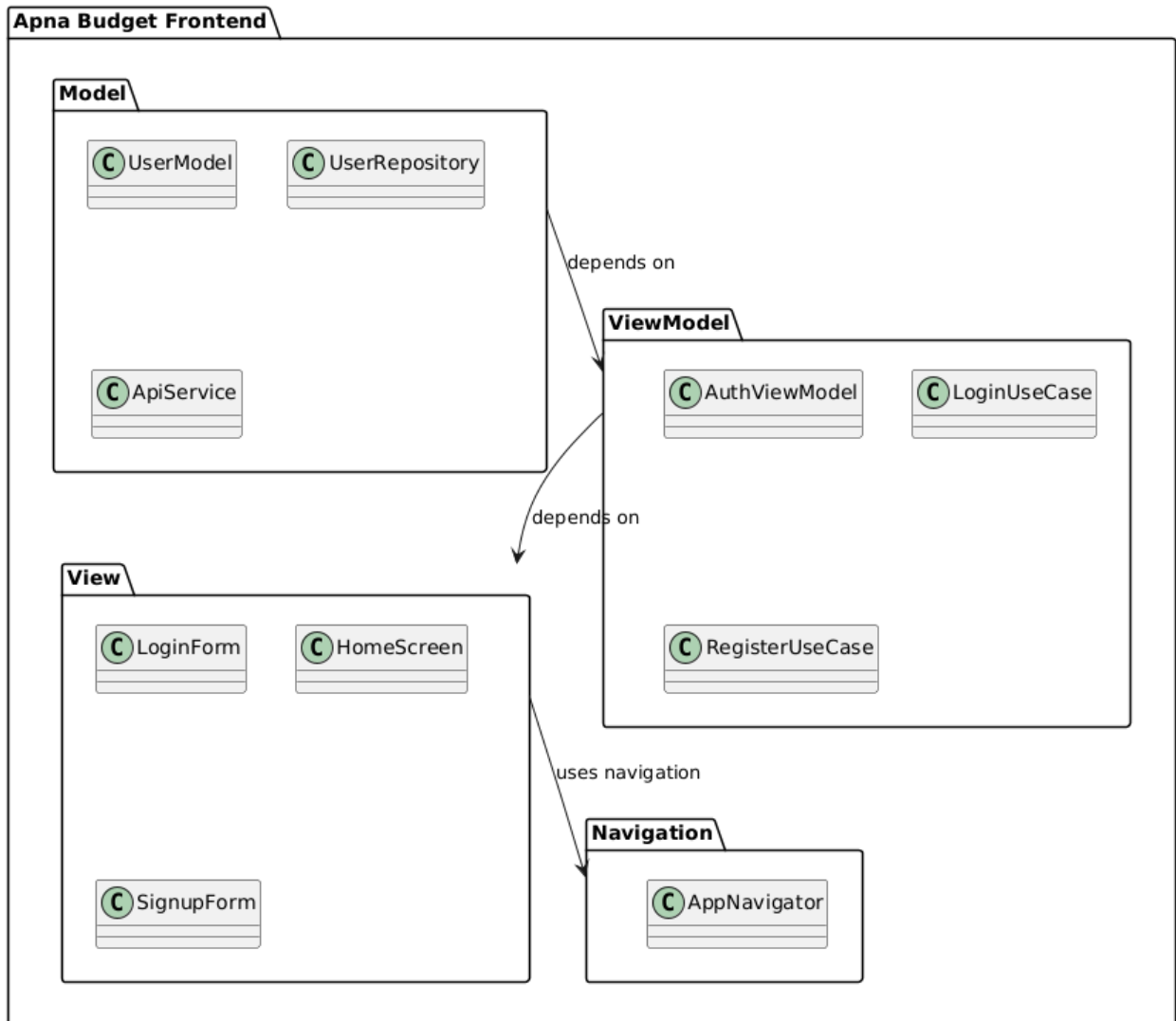


Figure 3.4: Package Diagram of Apna Budget - Frontend

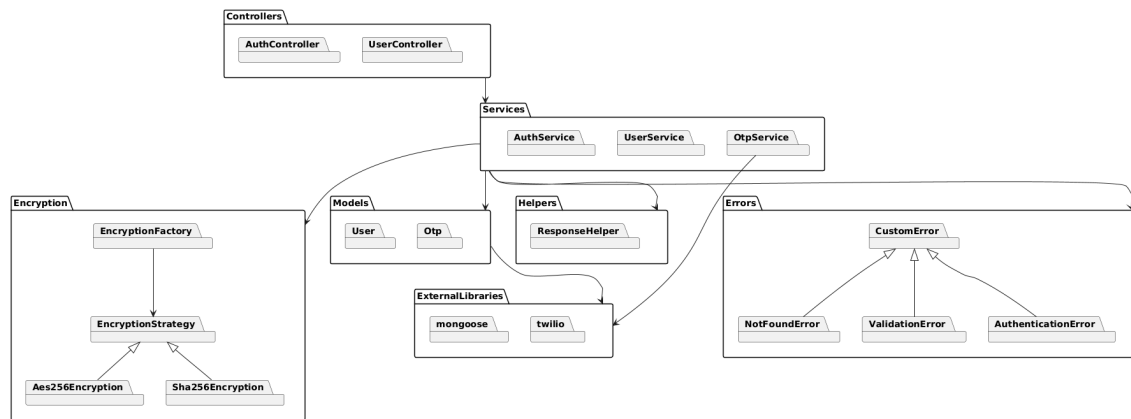


Figure 3.5: Package Diagram of Apna Budget - Backend

- Class Diagram



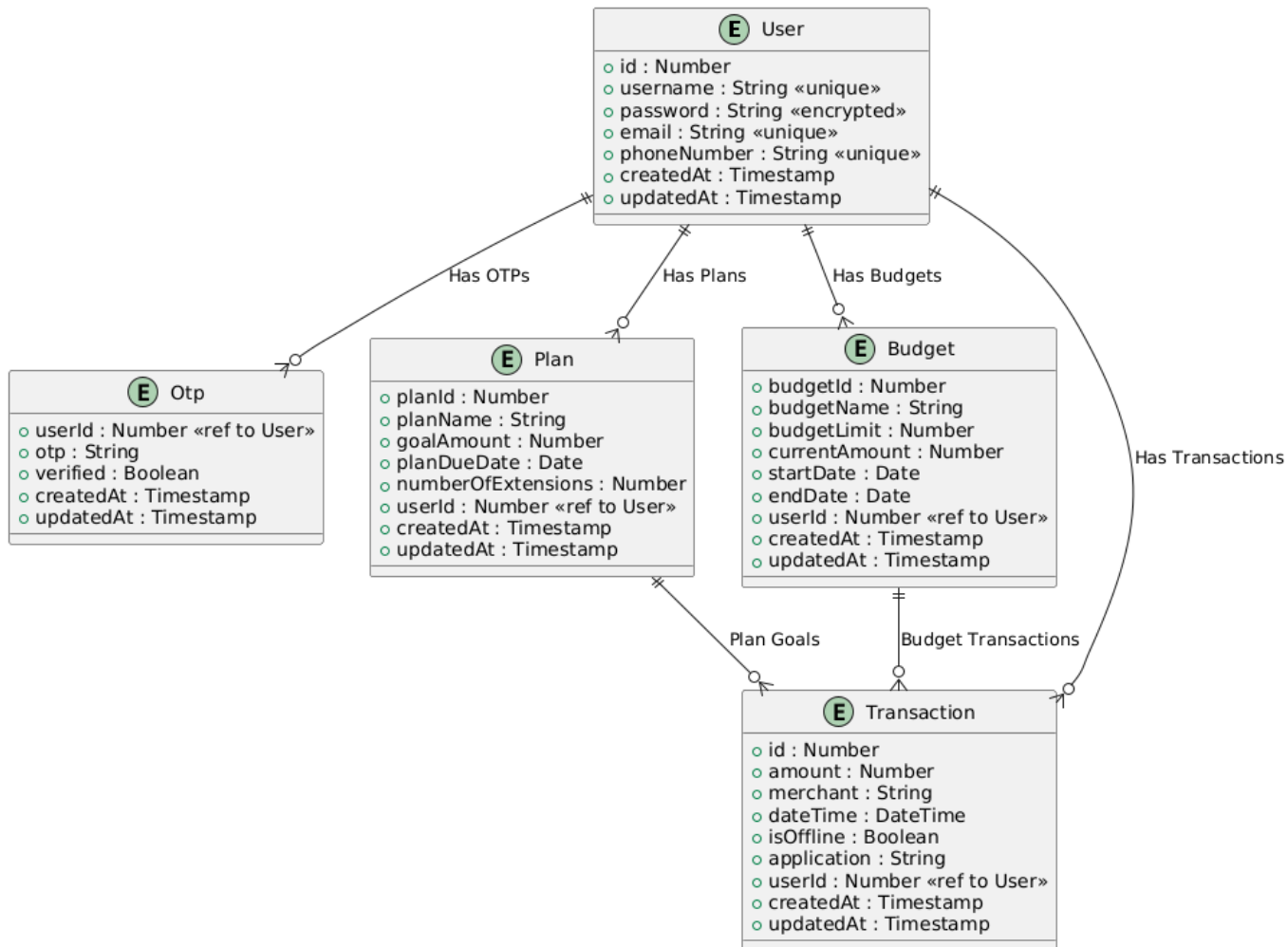


Figure 3.8: Data Model Diagram of Apna Budget